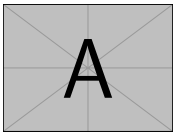
MetaCode

An LLM-Orchestrated Framework for Template-Driven Meta-Optics Design and Simulation

Architecture, LLM Reasoning, MCP-Style Tooling,
Computational Photonics, and Scientific Workflow



Technical Research Report

Repository: `MetaCode / MetaOpticsAI`

Package version: `metaopticsai 0.2.0`

October 6, 2026

Abstract

MetaCode (package name `metaopticsai`) is an LLM-orchestrated research prototype for *template-driven* meta-optics design. A Groq-hosted large language model plans a short, bounded sequence of tool calls against a verified toolbox of physics primitives; those primitives are backed by a vendored copy of the TensorFlow-based RCWA solver *metabox3* and by a dependency-free analytical fallback. Numeric quantities in the agent's final answer are produced by the solver, not by the LLM.

This report documents the repository as it exists on disk. The intent is not promotional: every architectural claim is grounded in a specific source file, and functionality that is not implemented is tagged explicitly as [FUTURE] or [PARTIAL]. The report covers the LLM layer (`llm/`), the in-process MCP-shaped tool layer (`mcp_server/`), the scientific core (`src/metaopticsai/`), the second, broader tool/schema layer (`src/metaopticsai/tools/`), the vendored RCWA backend, and the end-to-end demonstration in `examples/demo.py`. A dedicated chapter traces the exact tool-calling loop from a user prompt to a reportable design and metric set.

Status tag legend. [IMPLEMENTED] verified in source; [CONFIGURED] represented in configuration/registry but not exercised in the primary workflow; [PARTIAL] partially implemented; [EXPERIMENTAL] research-grade/unstable; [INFERRED] interpretation of code relationships; [FUTURE] suggested, not currently implemented.

Preface

This document is written as a reference manual for researchers and engineers who want to understand, extend, or validate the MetaCode codebase. It assumes basic familiarity with Python, Pydantic data models, meta-optics terminology (metasurface, metalens, unit cell, Jones vector), and the OpenAI/Groq style of function-calling. Mathematical content is kept to that which is actually invoked by the repository: focusing-phase profiles, Fraunhofer propagation, Strehl ratio, Zernike decomposition, and an abstract view of the RCWA problem that the vendored `metabox3` package solves.

Where the code departs from what documentation or module docstrings imply, this report records what the code actually does. Where the code and the `README.md` disagree, the code wins.

Contents

Abstract **2**

Preface **3**

1 Introduction **12**
1.1 Motivation . . . 12
1.2 What MetaCode Is . . . 12
1.3 What MetaCode Is Not . . . 13
1.4 Document Structure . . . 13

2 Meta-Optics Background **15**
2.1 Metasurfaces and Metalenses . . . 15
2.2 The Unit Cell and the Local Phase Response . . . 15
2.3 The Metalens Phase Profile . . . 16
2.4 Propagation and Point Spread Function . . . 16
2.5 Strehl Ratio and Zernike Decomposition . . . 17
2.6 Why RCWA? . . . 17
2.7 Where MetaCode's Mathematics Stops . . . 17

3 System Overview **19**
3.1 The Three Layers . . . 19
3.2 What Flows Between Layers . . . 19
3.3 Determinism Boundary . . . 21
3.4 Bounded-Loop Semantics . . . 21

4 Repository Architecture **22**
4.1 Top-Level Layout . . . 22
4.2 Package Map . . . 22
4.3 Directory Overview Table . . . 22
4.4 Import Discipline . . . 22
4.5 Vendor Boundary . . . 24

5 Core Source Architecture **26**
5.1 Domain Layer . . . 26
5.1.1 `domain/design.py` – `MetalensDesignParams` . . . 26

5.1.2 `domain/results.py` – result dataclasses . . . 26
5.1.3 `domain/handles.py` – `Artifact` . . . 27
5.1.4 `domain/incidence.py`, `materials.py`, `state.py` . . . 27
5.2 Configuration Layer . . . 27
5.3 Physics Layer . . . 27
5.3.1 Backend ABC . . . 27
5.3.2 The three backends . . . 28
5.3.3 High-level facades . . . 28
5.4 Phase Layer . . . 28
5.5 Analysis Layer . . . 29
5.6 Optimization Layer . . . 29
5.7 Fabrication Layer . . . 29
5.8 Scientific Tool Layer (`src/metaopticsai/tools/`) . . . 30
5.8.1 `BaseTool` and `ToolRegistry` . . . 30
5.8.2 The composition root . . . 30
5.8.3 Pydantic schemas . . . 30
5.9 Utility Layer . . . 31

6 LLM Architecture **32**
6.1 Package Layout . . . 32
6.2 Model and Configuration . . . 32
6.3 System Prompt . . . 32
6.4 Tool-Calling Loop . . . 33
6.4.1 Entry point . . . 33
6.4.2 Loop body . . . 33
6.4.3 What the LLM sees . . . 33
6.5 AgentResult . . . 34
6.6 LLM Architecture Diagram . . . 34
6.7 What the LLM Layer Does Not Do . . . 34

7 MCP Architecture **36**
7.1 Positioning . . . 36
7.2 Package Layout . . . 36
7.3 The Six Verified Tools . . . 36
7.4 Determinism and Error Handling . . . 37
7.5 Templates as Ground Truth . . . 37
7.6 The Workflow Guide . . . 37
7.7 The Simulator Wrapper . . . 38
7.8 MCP Architecture Diagram . . . 38
7.9 Extension Model . . . 38

8 The AI-Driven Workflow **40**
8.1 The Canonical Five-Stage Procedure . . . 40

8.2 Walkthrough: What `run_agent` Actually Does . . . 40
8.3 What the User Sees . . . 41
8.4 What the LLM Can and Cannot Do . . . 41
8.5 Dry-Run Mode . . . 42

9 Meta-Optics Computational Framework 43
9.1 Overview . . . 43
9.2 The Simulation Backend ABC . . . 43
9.3 AnalyticalBackend . . . 43
9.4 MetaboxBackend . . . 44
9.4.1 Unit-cell construction . . . 44
9.4.2 Simulation configuration . . . 44
9.4.3 Sweep . . . 44
9.4.4 Propagation . . . 44
9.4.5 Guarded availability . . . 45
9.5 S4Backend (Stub) . . . 45
9.6 High-Level Facades . . . 45
9.7 Material Index Lookup . . . 45
9.8 Scientific Computation Pipeline Diagram . . . 45

10 Design and Simulation Pipeline 47
10.1 Pipeline Boundaries . . . 47
10.2 Stages in Depth . . . 47
10.2.1 Stage 1 – Template selection . . . 47
10.2.2 Stage 2 – Parameter adaptation . . . 48
10.2.3 Stage 3 – Validation . . . 48
10.2.4 Stage 4 – Simulation . . . 48
10.2.5 Stage 5 – Interpretation . . . 49
10.3 Beyond Simulation: What the Agent Does Not Do . . . 49
10.4 A Scientist-Style Pipeline . . . 49

11 End-to-End Case Studies 51
11.1 Case Study 1 – A Green Visible Metalens . . . 51
11.2 Case Study 2 – A UV Metalens with GaN . . . 52
11.3 What Breaks the Workflow . . . 52
11.4 Reading the Trace . . . 53

12 Data and Control Flow 54
12.1 Data Classes Across the Stack . . . 54
12.2 Numeric Data Flow . . . 54
12.3 Control Flow in the Agent Loop . . . 55
12.4 Error Flow . . . 56
12.5 Dependency Graph . . . 56

13 Software Engineering Analysis **57**
13.1 Architecture . . . 57
13.2 Modularity . . . 57
13.3 Extensibility . . . 58
13.4 Error Handling . . . 58
13.5 Logging . . . 58
13.6 Testing . . . 58
13.7 Reproducibility . . . 59
13.8 Performance . . . 59
13.9 Security . . . 59
13.10 Portability . . . 60

14 Validation and Reproducibility **61**
14.1 What Needs to Be Validated . . . 61
14.2 Validating the Physics Layer . . . 61
14.3 Validating the Agent . . . 62
14.4 Reproducibility Checklist . . . 62
14.5 Known Reproducibility Hazards . . . 62

15 Limitations **63**
15.1 Current Limitations (A) . . . 63
15.2 Missing Capabilities (B) . . . 63
15.3 Architectural Limitations (C) . . . 64
15.4 Scientific Limitations (D) . . . 64
15.5 Performance Limitations (E) . . . 64
15.6 LLM Limitations (F) . . . 65
15.7 MCP Limitations (G) . . . 65
15.8 Reproducibility Limitations (H) . . . 65

16 Future Directions **66**
16.1 Short-Term Engineering Work . . . 66
16.2 Short-Term Scientific Work . . . 66
16.3 Research Directions . . . 67
16.4 Mapping Future Work to Current Files . . . 67

17 Conclusion **69**

A Repository Tree **70**

B MCP Tool Reference **72**
B.1 The Six LLM-Visible MCP Tools . . . 72
B.1.1 `list_templates` . . . 72
B.1.2 `get_template` . . . 72
B.1.3 `get_workflow_guide` . . . 72

B.1.4 validate_design . . . 73
B.1.5 run_simulation . . . 73
B.1.6 get_optimization_tips . . . 74
B.2 The Broader Scientific Tool Catalogue ([CONFIGURED]) . . . 74
B.2.1 run_rcwa_sweep . . . 74
B.2.2 get_fdtd_library . . . 74
B.2.3 list_materials . . . 74
B.2.4 get_material_index . . . 75
B.2.5 generate_phase_mask . . . 75
B.2.6 analyze_psf . . . 75
B.2.7 analyze_mtf . . . 75
B.2.8 zernike_decompose . . . 75
B.2.9 propagate_field . . . 75
B.2.10 compute_max_intensity / compute_center_intensity / compute_mtf_volume 75
B.2.11 export_gds . . . 75
B.2.12 optimize_metalens . . . 76
B.2.13 train_metamodel . . . 76
B.2.14 get_job . . . 76
B.2.15 list_artifacts . . . 76

C Configuration **77**
C.1 Environment Variables . . . 77
C.2 config/.env.example . . . 77
C.3 Settings Singleton . . . 77
C.4 RCWA Knobs in Practice . . . 78
C.5 Groq Model Choice . . . 78

D API Reference **79**
D.1 llm/ . . . 79
D.1.1 llm.groq_client . . . 79
D.1.2 llm.agent . . . 79
D.2 mcp_server/ . . . 79
D.2.1 mcp_server.tools . . . 79
D.2.2 mcp_server.templates . . . 79
D.2.3 mcp_server.workflow . . . 80
D.2.4 mcp_server.simulator . . . 80
D.3 src/metaopticsai/ . . . 80
D.3.1 metaopticsai.domain . . . 80
D.3.2 metaopticsai.configs.settings . . . 80
D.3.3 metaopticsai.physics . . . 80
D.3.4 metaopticsai.physics.backends . . . 80
D.3.5 metaopticsai.phase . . . 81
D.3.6 metaopticsai.analysis . . . 81

D.3.7 `metaopticsai.optimization` 81
D.3.8 `metaopticsai.fabrication.gds.GDSEngine` 81
D.3.9 `metaopticsai.tools` 81
D.3.10 `metaopticsai.utils` 82

E Reproducibility Checklist **83**
E.1 Environment 83
E.2 Fixing the Agent 83
E.3 Fixing the Physics 83
E.4 Capturing the Session 84
E.5 Verifying Numeric Faithfulness 84
E.6 Minimal Reproducer Script 84

List of Figures

3.1 High-level dataflow in the shipping agent workflow. The user's prompt enters at the top; each layer forwards structured JSON to the next. The LLM only sees Pydantic summaries; raw numpy arrays never leave the physics layer. 20

4.1 Logical dependency graph inside `src/metaopticsai/`. Arrows point from caller to callee. The vendored `metabox3` is imported by only two modules (`physics/backends/metabox.py` and `physics/materials.py`). 23

6.1 The LLM loop as actually implemented in `llm/agent.py`. The LLM sees the trimmed view; the `AgentResult` returned to the caller preserves the full one. . . . 34

7.1 The MCP-shaped tool catalogue. Solid arrows denote dispatch; vertical adjacency reflects the staged dependency implied by the system prompt and workflow guide. 39

8.1 The five-stage design workflow with the single conditional retry edge. The iteration cap stops the loop at five tool calls regardless. 41

9.1 The scientific computation pipeline for a diameter sweep. The registry's `get("auto")` chooses metabox when available and analytical otherwise. 46

10.1 A plausible scientist-driven pipeline using the broader tool catalogue. Each node writes an artifact handle; edges carry the handle, not the payload. No current LLM transport drives this pipeline ([CONFIGURED]). 50

12.1 Numeric data flow with every size-reducing step shown. The LLM only ever observes the `sweep_head` + scalar summary, not the full sweep. 55

12.2 Package dependency graph for the shipping repository. The two routes from the top – demo via LLM, demo direct – share the MCP tool layer and the physics core. 56

List of Tables

4.1 Repository directories and their roles. 23
4.2 Vendored metabox3 sub-modules used by MetaCode. 25

5.1 Fields of `MetalensDesignParams`. 26
5.2 Simulation backends. 28
5.3 Tools registered by `build_tool_registry`. 30

6.1 Environment variables consulted by the LLM layer. 32

7.1 Verified MCP-shaped tool catalogue (`mcp_server/tools.py`). 37

10.1 Pipeline boundaries and data shapes. 47

12.1 Principal data objects, their type, and lifetime. 54

13.1 Modules and their public surface. 57

16.1 Future-work entries and the files most likely to change. 68

C.1 Environment variables. 77

Chapter 1

Introduction

1.1 Motivation

Meta-optics – the engineering of sub-wavelength arrays of resonant scatterers to shape the complex amplitude of light – has, in a decade, moved from table-top demonstrations [11, 6, 5] toward large-aperture devices intended for real optical systems. As the design space has widened, end-to-end workflows have become increasingly multi-stage: a designer picks a wavelength and a material, chooses a meta-atom geometry, probes the geometry's phase/transmission response with a numerical electromagnetic solver (RCWA [9] or FDTD), discretizes a target phase profile against the resulting library, and finally emits a GDSII layout fit for lithography. Each of these stages has its own tool-chain and its own failure modes.

Large language models (LLMs) excel at *orchestration* and at answering questions in natural language, but they do not solve Maxwell's equations. A design agent that lets the LLM invent numeric field values is unsafe. MetaCode is one answer to that constraint: the LLM is used strictly to *plan* and to *select* a short sequence of calls into a verified toolbox; every scientific number that reaches the user originates in a deterministic call to the vendored `metabox3` RCWA solver or to a clearly-labelled analytical fallback.

1.2 What MetaCode Is

MetaCode (package `metaopticsai`) is a research prototype that implements:

- A [IMPLEMENTED] Python package `src/metaopticsai/` containing a domain layer (Pydantic design parameters, incidence specs, result dataclasses), a physics layer with pluggable simulation backends, phase-profile generators, Fourier-optics-based analysis routines, three optimizer strategies, and a GDSII exporter.
- A [IMPLEMENTED] `llm/` package containing a Groq client wrapper and a tool-calling loop capped at five iterations.
- A [IMPLEMENTED] `mcp_server/` package exposing six verified tools in the JSON-Schema shape expected by both the Groq/OpenAI tool-calling API and the Model Context Protocol (MCP) [1]. The tools are dispatched *in-process* by the agent; no stdio MCP process is spawned in the current repository.

- A [CONFIGURED] second tool layer at `src/metaopticsai/tools/` with a richer `BaseTool`/`ToolRegistry` API and Pydantic input/output schemas for roughly a dozen scientific tools (RCWA sweeps, phase-mask generation, PSF/MTF/Zernike analysis, propagation, materials lookup, GDS export, optimization, metamodel training). This layer is composed in `build_tool_registry` but is not driven by the current LLM agent – it is architecturally positioned for a future, broader MCP server.
- A [IMPLEMENTED] vendored copy of the `metabox3` [4] TensorFlow-based RCWA simulator at `src/metaopticsai/vendor/metabox3/`, used as the high-fidelity backend.
- A [IMPLEMENTED] runnable demo (`examples/demo.py`) exercising the whole agent/tool/physics stack, both as a dry run (no LLM) and as a live run (LLM-driven).

1.3 What MetaCode Is Not

Equally important is what the repository does *not* implement in its primary workflow. Clarifying this up front avoids misreading the agent's capabilities later.

- [FUTURE] **No inverse design in the agent loop.** The LLM-driven workflow is forward-only: it starts from a verified template, mutates a small handful of parameters, validates, simulates, and reports. Gradient-based inverse design is present in `metaopticsai.optimization.gradient` but is called only outside the agent loop, and the `README` is explicit about this limitation.
- [FUTURE] **No stdio MCP process.** `mcp_server/tools.py` returns JSON-Schema tool descriptors in the MCP/OpenAI shape, but the `dispatch()` function is called directly from `llm/agent.py` in the same Python process. Spinning up a real stdio MCP server is called out in the `mcp_server/README.md` as a ∼30-line future change.
- [PARTIAL] **No multi-agent orchestration.** There is exactly one agent, one chat loop, and one tool registry; there is no planner/critic split, no RAG, no vector store.
- [PARTIAL] **The S4 backend is a stub.** `S4Backend` declares the interface and advertises an availability check, but `simulate_unit_cell`, `sweep_unit_cells`, and `propagate_field` all raise `NotImplementedError`.
- [PARTIAL] **Surrogate optimization is a skeleton.** `SurrogateOptimizer` documents the intended call into `metabox3.modeling.load_metamodel`, but its `optimize()` returns a zero-FOM `status: not_implemented`.
- [PARTIAL] **The broader tool layer** at `src/metaopticsai/tools/` compiles and runs via `build_tool_registry`, but no shipping MCP transport currently binds it to an LLM.

1.4 Document Structure

Chapter 2 summarises the physics vocabulary that the code assumes. Chapter 3 gives a high-level view of the system. Chapter 4 walks the directory tree. Chapters 5–7 dissect the three principal

code layers. Chapter 8 shows how the layers compose into a single tool-calling conversation. Chapters 9 and 10 describe the computational framework and the design pipeline. Chapter 11 presents two annotated case studies. Chapter 12 summarises data and control flow. Chapters 13 and 14 audit the software engineering and reproducibility story. Chapters 15 and 16 enumerate observed limitations and research directions. Chapter 17 concludes.

Five appendices provide (A) a cleaned repository tree, (B) a tool-by-tool reference for both the `mcp_server/` and `src/metaopticsai/tools/` layers, (C) a configuration summary, (D) a function-/class-level API reference, and (E) a reproducibility checklist.

Chapter 2

Meta-Optics Background

This chapter establishes only the physical vocabulary that MetaCode actually uses. Nothing here anticipates functionality that is not exercised by the repository.

2.1 Metasurfaces and Metalenses

A *metasurface* is a planar array of sub-wavelength scatterers – *meta-atoms* – whose local resonant response controls the complex amplitude of a transmitted or reflected wave [11, 6, 2]. When a metasurface is engineered so that its position-dependent phase matches that of a converging spherical wavefront, the result is a flat *metalens* [5].

MetaCode's templates (see `mcp_server/templates.py`) encode three concrete starting points: TiO_2 pillars for visible light at 532 nm, silicon cylinders for the near-infrared at 850 nm, and GaN pillars for the near-ultraviolet at 405 nm. Each template fixes a pillar material, a nominal pillar height, a square unit-cell period, and a diameter sweep range known from the literature to yield full 2π phase coverage.

2.2 The Unit Cell and the Local Phase Response

A meta-atom is modelled as a cylindrical dielectric pillar embedded in a square unit cell of pitch p, standing on a substrate of index n_{sub} and surrounded by air (index 1). The local transmission coefficient $t(d;\lambda)$ at wavelength λ is a complex scalar that depends on the pillar diameter d, its height h, the pillar material refractive index $n_{\text{p}}(\lambda)$, and the unit-cell geometry. Its modulus controls the local transmission efficiency; its argument controls the local phase imprint:

$$t(d;\lambda) = |t(d;\lambda)|\, e^{\,i\,\varphi(d;\lambda)}. \tag{2.1}$$

MetaCode exposes two concrete models for Eq. (2.1):

- [IMPLEMENTED] A heuristic *analytical* model (`AnalyticalBackend`) with the leading-order thin-film approximation

$$\varphi_{\max} = \frac{2\pi\,(n_{\text{p}} - 1)\,h}{\lambda}, \qquad \varphi(d) = \varphi_{\max}\,\sigma\!\left(\frac{d - 50}{d}\right), \tag{2.2}$$

where $\sigma(\cdot)$ is a logistic function that smooths the phase versus diameter curve. This model is not a substitute for RCWA; it is used for CI, notebook iteration without TensorFlow, and quick feasibility checks.

- [IMPLEMENTED] A full *RCWA* solve via `MetaboxBackend`, delegating to `metabox3.rcwa.simulate_paramet` [4, 9]. The adapter instantiates a `ProtoUnitCell` with a single `Circle` shape whose radius is marked variable (a metabox `Feature`), configures the simulator (XY harmonics, resolution, mini-batch size), and extracts the complex zeroth-order transmission.

2.3 The Metalens Phase Profile

For a thin focusing lens placed at $z = 0$, the diffraction-limited phase profile that converts a normally-incident plane wave into a converging spherical wave of focal length f is

$$\phi(r) \;=\; \frac{2\pi}{\lambda}\left(f - \sqrt{r^2 + f^2}\right), \tag{2.3}$$

which MetaCode writes modulo 2π to produce a Fresnel zone lens (see `metaopticsai.phase.profiles.fresnel_zon`

$$\phi_{\mathrm{FZL}}(r) \;=\; \left[2\pi \;-\; \frac{2\pi}{\lambda}(\sqrt{r^2 + f^2} - f)\right] \mod 2\pi. \tag{2.4}$$

Two other canonical phase elements are implemented:

- [IMPLEMENTED] An *axicon* with cone half-angle α,

$$\phi_{\mathrm{axicon}}(r) = [\,k\,(R - r)\tan\alpha\,] \mod 2\pi, \qquad k = \tfrac{2\pi}{\lambda}, \tag{2.5}$$

  which generates a Bessel-beam over a depth $z_{\max}$ set by the cone angle and aperture.

- [IMPLEMENTED] A *spiral phase plate* of topological charge ℓ,

$$\phi_{\mathrm{SPP}}(\theta) = [\ell\,\theta + \ell\,\pi] \mod 2\pi, \tag{2.6}$$

  which carries orbital angular momentum $\ell\hbar$ per photon.

A continuous phase mask is then [IMPLEMENTED] quantised to an integer label grid using `metaopticsai.phase.quantization.quantize`; the number of levels is a free parameter (default 8). The quantised mask, together with a sweep's *phase→diameter* inverse table, drives the GDS layout step.

2.4 Propagation and Point Spread Function

MetaCode uses the Fraunhofer approximation for far-field focus estimation [3]. Given a complex pupil $U(x, y) = A(x, y)\,e^{\,i\phi(x,y)}$, the normalized far-field intensity is

$$I_{\mathrm{far}}(u, v) \;=\; \frac{\left|\mathcal{F}\{A\,e^{i\phi}\}\right|^2}{\max\left|\mathcal{F}\{A\,e^{i\phi}\}\right|^2}, \tag{2.7}$$

implemented in `analysis/psf.py` and `analysis/farfield.py` with a 4× zero-padded 2-D FFT.

A finite focal distance scales the pupil coordinates by $\lambda f/(N_{\text{pad}}\, p_x)$, matching the usual relation between pupil and image samplings.

[IMPLEMENTED] A second propagation path exists in `MetaboxBackend.propagate_field`, which hands the field to `metabox3.propagation` and uses an angular-spectrum transfer function; this is the preferred path when TensorFlow is available.

2.5 Strehl Ratio and Zernike Decomposition

The *Strehl ratio* is the ratio between the on-axis PSF peak of the aberrated system and that of an unaberrated identical-aperture system; the Maréchal criterion [8] is $S \geq 0.8$:

$$S \;=\; \frac{\max\left|\mathcal{F}\{A\, e^{i\phi}\}\right|^2}{\max\left|\mathcal{F}\{A\}\right|^2}. \tag{2.8}$$

`compute_strehl_ratio` in `analysis/strehl.py` evaluates Eq. (2.8) by two padded FFTs over the same aperture.

The repository also provides a Noll-indexed Zernike decomposition [10] up to the user-selected number of terms (`analysis/zernike.py`). For a mask sampled on (ρ, θ) with $\rho \leq 1$,

$$\phi(\rho,\theta) \;=\; \sum_{j=1}^{N_{\text{terms}}} c_j\, Z_j(\rho,\theta), \qquad c_j \approx \frac{1}{N_{\text{px}}} \sum_{(\rho_k,\theta_k)\in\text{disk}} \phi(\rho_k,\theta_k) Z_j(\rho_k,\theta_k), \tag{2.9}$$

with Noll-indexed names (defocus, tilt, coma, spherical, …) reported for the top five coefficients by magnitude.

2.6 Why RCWA?

MetaCode uses rigorous coupled-wave analysis as its workhorse because (i) unit-cell response at a single wavelength maps naturally onto the problem of *periodic layered structures*, exactly what RCWA was designed for [9, 7]; and (ii) RCWA is faster than FDTD and amenable to batched evaluation on GPUs, which is how `metabox3` [4] is implemented (TensorFlow backend, mini-batched sweeps).

[FUTURE] FDTD remains the second obvious choice for validation. The repository ships a static reference library of FDTD-tabulated phase-vs-diameter curves at five wavelengths (`physics/fdtd.py`) but does not run FDTD itself.

2.7 Where MetaCode's Mathematics Stops

The theoretical scope is deliberately narrow:

- Scalar diffraction only. Vector near-field analysis past the RCWA output is not performed outside what `metabox3` itself does internally.

- Normal incidence by default. The `IncidenceSpec` dataclass admits off-normal (θ, ϕ) parameters, but every tool in the LLM-driven loop sets them to zero.
- x-polarised illumination. The Jones vector defaults to $(1, 0)$ throughout.
- Monochromatic design. Only one wavelength is used per sweep. Chromatic analysis is out of scope.

Chapter 3

System Overview

3.1 The Three Layers

MetaCode is organised around three layers that communicate through well-defined, mostly JSON-shaped boundaries:

1. **LLM orchestration** (`llm/`). A thin wrapper around the Groq SDK plus a tool-calling loop. Owns the conversation state; does not know what the tools do.
2. **MCP-shaped verified tools** (`mcp_server/`). A fixed registry of six functions that enforce a canonical five-stage design workflow and gate every call into physics through a Pydantic validation step.
3. **Scientific core** (`src/metaopticsai/`). The domain data model, the physics backends, phase profiles, analysis, optimizers, fabrication, and the vendored `metabox3` RCWA solver.

A *second* tool layer at `src/metaopticsai/tools/` provides a richer `BaseTool`/`ToolRegistry` API over the scientific core. It is [CONFIGURED] – it compiles, has strict Pydantic schemas, and is composed by `build_tool_registry` – but no LLM transport in the shipping repository drives it.

3.2 What Flows Between Layers

- **User → LLM.** A free-form natural language prompt, concatenated with a strict system prompt embedded in `llm/agent.py` that dictates the five-stage procedure.
- **LLM → MCP tool.** A JSON-encoded tool call conforming to the OpenAI/Groq function-calling schema. `llm/agent.py` decodes `function.arguments` from JSON, catches malformed JSON, and forwards `(name, args)` to `mcp_server.tools.dispatch`.
- **MCP tool → physics.** Dict→Pydantic conversion through `MetalensDesignParams.model_validate` is the single hard boundary. No invalid parameter object ever reaches `metabox3`.
- **Physics → MCP tool.** Backends return plain-Python dataclasses (`RCWASweepResult`, `UnitCellResult`, `FieldResult`) populated with `numpy.ndarrays`. No TensorFlow tensors leak upward.

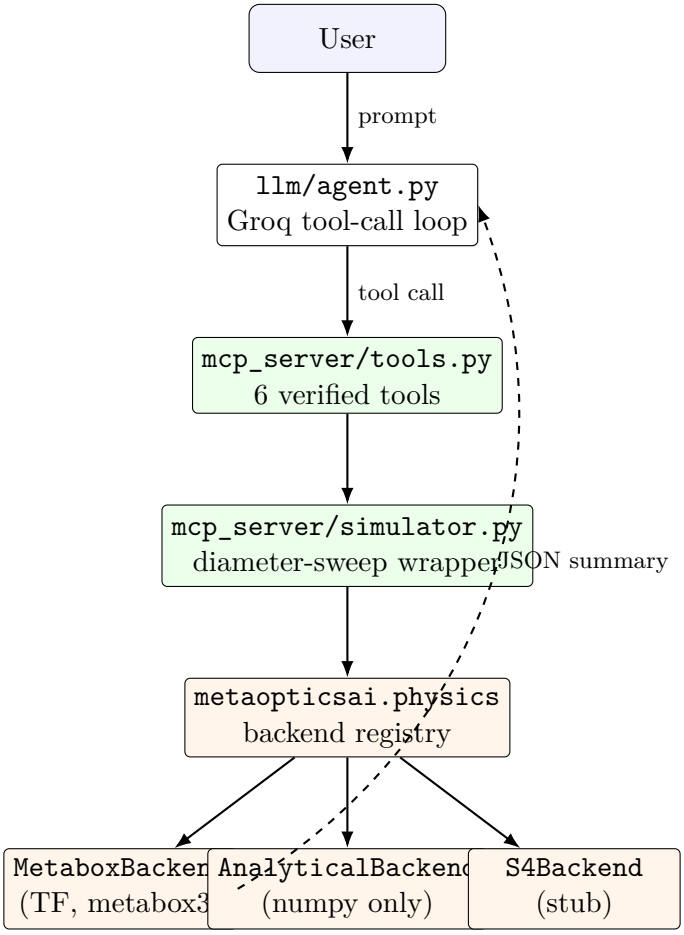


Figure 3.1: High-level dataflow in the shipping agent workflow. The user's prompt enters at the top; each layer forwards structured JSON to the next. The LLM only sees Pydantic summaries; raw numpy arrays never leave the physics layer.

- **MCP tool → LLM.** Returns a dict of the form `{"ok": bool, ...}`, with the full sweep arrays stripped into a `sweep_head` first five entries plus scalar summary metrics. The full arrays live in a private `"_full"` key kept locally for the caller.
- **LLM → User.** A short textual summary of the final design parameters and metrics.

3.3 Determinism Boundary

Because MetaCode's correctness depends on where in the pipeline numbers are generated, the *determinism boundary* is worth stating precisely:

Non-deterministic region. LLM output is non-deterministic; the LLM may pick different tools, use different parameter values from the templates, or write different natural-language summaries on different runs.

Deterministic region. Everything beneath `dispatch()` is deterministic given a fixed input dict and a fixed backend. Pydantic validation, `numpy.linspace`, the analytical closed-form backend, and the `metabox3` RCWA solve (modulo TF non-determinism in reductions) are all deterministic.

This is why the `README`'s guarantee – "every metric traces back to a deterministic call into `metabox3`" – is defensible: no numeric field value is parsed back out of an LLM message, nor is any physical constant chosen by the LLM.

3.4 Bounded-Loop Semantics

The agent in `llm/agent.py` is capped by `AGENT_MAX_ITERATIONS` (default 5, clamped to the interval $[1, 5]$). Each iteration is one *round-trip* with Groq:

1. Send the current message list.
2. If the response has no `tool_calls`, return the final text.
3. Otherwise, execute each requested tool and append a `role="tool"` message per result.

The iteration cap is a safety feature. Combined with the system prompt's instruction to stop after one round of optimisation tips, the agent is unlikely to spend more than three calls on any single design request.

Chapter 4

Repository Architecture

4.1 Top-Level Layout

The repository root contains the following project-relevant entries:

- `config/` – a `.env.example` and a (git-ignored) `.env` for Groq credentials and model settings.
- `llm/` – Groq client wrapper, tool-calling agent, `python -m llm` entry point.
- `mcp_server/` – MCP-shaped verified tools, templates, workflow guide text, simulator wrapper.
- `src/metaopticsai/` – the scientific package.
- `examples/demo.py` – a dry-run and live-run demo.
- `notebooks/Implementation.ipynb` – a 646 kB exploration notebook (not analysed in this report).
- `docs/` – the pre-existing `METAOPTICS_REPORT.pdf`, and (from this task) the Overleaf-ready LaTeX project at `docs/overleaf_latex/`.
- `requirements.txt`, `README.md`, `LICENSE`, `.gitignore`.

4.2 Package Map

4.3 Directory Overview Table

Table 4.1 summarises each directory's purpose, roughly in order of dependency.

4.4 Import Discipline

The code enforces a strict dependency gradient from pure data to I/O:

- `domain/` depends on nothing within the package. Its handles module (`domain/handles.py`) is explicitly documented as a "pure leaf".

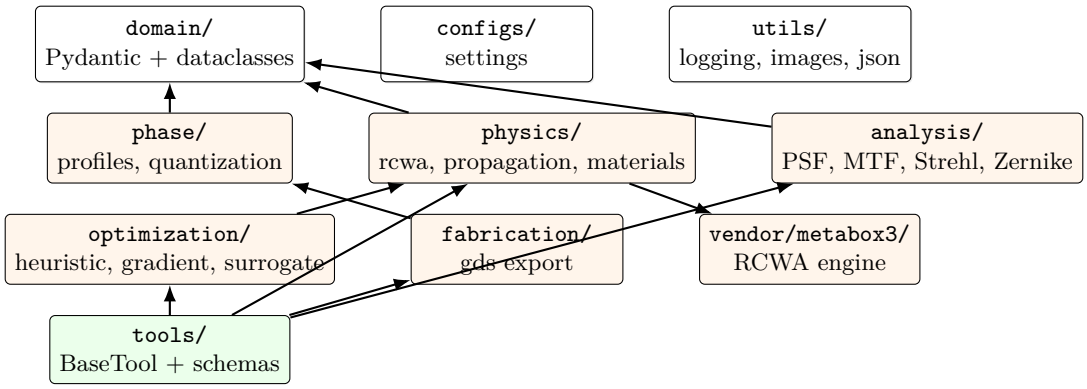


Figure 4.1: Logical dependency graph inside `src/metaopticsai/`. Arrows point from caller to callee. The vendored `metabox3` is imported by only two modules (`physics/backends/metabox.py` and `physics/materials.py`).

Table 4.1: Repository directories and their roles.

Path	Role
`llm/`	Groq client; tool-calling loop; CLI.
`mcp_server/`	Verified MCP-shaped tool registry (6 tools), templates, workflow guide, simulator wrapper.
`src/metaopticsai/domain/`	Pure data: Pydantic design params, incidence spec, materials, artifacts, result dataclasses, design state.
`src/metaopticsai/configs/`	Pydantic-settings based project settings + path registry.
`src/metaopticsai/physics/`	High-level RCWA/propagation facades + backend registry.
`src/metaopticsai/physics/backends/`	`SimulationBackend` ABC, analytical heuristic, metabox adapter, S4 stub.
`src/metaopticsai/phase/`	Fresnel-zone-lens, axicon, SPP phase profiles + quantiser.
`src/metaopticsai/analysis/`	PSF, MTF, Strehl, Zernike, encircled energy, far-field.
`src/metaopticsai/optimization/`	Optimizer ABC + heuristic/gradient/surrogate implementations.
`src/metaopticsai/fabrication/`	GDS export via gdsfactory.
`src/metaopticsai/tools/`	Richer `BaseTool`/`ToolRegistry` API with Pydantic I/O schemas ([CONFIGURED]).
`src/metaopticsai/vendor/metabox3/`	Vendored metabox3 RCWA engine + material-index CSVs.
`src/metaopticsai/utils/`	Async helpers, numpy-aware JSON encoder, matplotlib-to-PNG utilities, logging.
`examples/`	`demo.py` dry/live-run harness.
`config/`	Secrets (git-ignored).
`notebooks/`	Exploratory notebook (not analysed).
`docs/overleaf_latex/`	This report.

- `configs/` depends on `pydantic-settings` and on nothing in `metaopticsai`.
- `physics/` depends on `domain/` and `configs/`, plus one lazy import of `vendor.metabox3` from a single backend file. The ABC in `physics/backends/base.py` explicitly instructs backends to defer heavy imports and never leak TF/torch tensors.
- `analysis/`, `phase/`, `fabrication/`, `optimization/` depend on `domain/` and (for `optimization/`) on `physics/`.
- `tools/` is the top layer and depends on all of `domain/`, `physics/`, `analysis/`, `phase/`, `fabrication/`, and `optimization/`.
- `llm/` depends only on `mcp_server/`; it has no direct scientific imports.

The $_{i}nit_{.}py of$ `metaopticsai` $explicitly references an import-linter contract (“follow the dependency rules in$ `pyproject.toml[tool.importlinte`

4.5 Vendor Boundary

The vendored metabox3 package is imported from exactly two locations:

- src/metaopticsai/physics/backends/metabox.py (unit cell and sweep RCWA; angular-spectrum propagation).
- src/metaopticsai/physics/materials.py (CSV-based complex refractive index via metabox3.rcwa.Mat

Both imports are lazy: they happen inside method bodies or inside is_available(), so cold-start cost of the top-level package is independent of whether TensorFlow is installed.

Table 4.2 lists the sub-modules that vendor/metabox3/ contains and the public symbols that the MetaCode adapters consume.

Table 4.2: Vendored metabox3 sub-modules used by MetaCode.

Sub-module	**What MetaCode uses from it**
`utils.py`	`Feature`, `Incidence`, `CoordType`, `ParameterType`.
`rcwa.py`	`Circle`, `Layer`, `UnitCell`, `ProtoUnitCell`, `SimConfig`, `simulate_parameterized_unit_cells`, `Material`.
`propagation.py`	`Field2D`, `get_transfer_function`, `propagate`.
`assembly.py`	`Metasurface`, `LensAssembly`, `FigureOfMerit`, `optimize_single_lens_assembly` (used only by the *gradient* optimiser, outside the agent).
`modeling.py`	`sample_protocell`, `create_and_train_model`, `load_metamodel` (used only by `train_metamodel` tool and surrogate stub).
`material_data/`	12 CSVs of complex refractive index tables (GaN, Si_ir, Si_visible, Si_3N_4, TiO_2, ZnO, ZrO_2, N-BK7, MoO_3, Nb_2O_5, quartz, sample).
`raster.py`, `expansion.py`, `export.py`, `metrics.py`	Not directly referenced by MetaCode's adapters.
`rcwa_tf/`	Internal metabox3 TensorFlow kernels.

Chapter 5

Core Source Architecture

This chapter examines src/metaopticsai/ module by module. The goal is to establish, for each unit, what it does, what it depends on, and where it is used.

5.1 Domain Layer

5.1.1 domain/design.py – MetalensDesignParams

A Pydantic v2 BaseModel that constitutes the *single source of truth* for a metalens design request. Its fields are listed in Table 5.1.

Table 5.1: Fields of MetalensDesignParams.

Field	Role	Validator
wavelength_nm	Design wavelength (nm).	(200, 20000)
focal_length_mm	Target focal length (mm).	> 0
diameter_mm	Aperture diameter (mm).	> 0
pillar_material	Literal["TiO2","Si","GaN","SiN"].	–
pillar_height_nm	Pillar height (nm).	(50, 3000)
min_diameter_nm	Smallest pillar diameter in sweep.	(10, 2000)
max_diameter_nm	Largest pillar diameter in sweep.	(20, 2000)
periodicity_nm	Square unit-cell pitch.	(100, 2000)
xy_harmonics	Pair of RCWA harmonic counts.	default $(3, 3)$
n_radial_pixels	Radial sampling (lens-scale).	$[5, 400]$
substrate_index	Substrate refractive index.	> 1.0
jones_vector	Polarisation Jones vector.	default $(1, 0)$

A cross-field validator (_check_geometry) enforces two non-trivial constraints: min_diameter_nm < max_diameter_nm, and max_diameter_nm $\leq$ periodicity_nm - 20 nm, the second ensuring at least 20 nm of wall between adjacent pillars.

Four @computed_fields expose derived quantities to downstream tools without storing them: numerical aperture, f-number, mean diameter, and aspect ratio. The LLM-visible validate_design MCP tool uses these to produce human-readable warnings when AR > 10 or NA > 0.85.

5.1.2 domain/results.py – result dataclasses

Five dataclasses carry simulation outputs across the physics boundary:

- UnitCellResult - a single t_x, t_y complex pair plus modulus, phase, elapsed time, backend name.
- RCWASweepResult - the sweep axis, amplitudes and unwrapped-and-zero-aligned phases, phase coverage ratio, mean transmission, elapsed time, backend.
- FieldResult - a 2-D intensity array plus peak and wavelength.
- PSFResult - a PSF with precomputed Strehl and FWHM.
- FOMResult - a scalar figure of merit with provenance (name, backend, iteration, metadata).

The dataclasses are *plain* because Pydantic v2 cannot validate numpy arrays efficiently. Each has a to_payload() that serialises arrays to lists for the artifact store.

5.1.3 domain/handles.py — Artifact

An immutable, frozen dataclass with a 12-hex-character UUID, an ArtifactKind tag (one of phase_mask, quantized_mask, rcwa_sweep, psf, mtf, zernike, field_2d, gds_file, metalens_design, metamodel, job_result), the raw payload (array/dict/Path), creation timestamp, and metadata.

Its summary() method returns a JSON-safe description containing shape, keys, or path - but *never* the payload bytes - so the LLM can be told "such an artifact exists" without receiving its contents. This is the mechanism that makes the handle-based protocol in src/metaoptics transport-agnostic.

5.1.4 domain/incidence.py, materials.py, state.py

- IncidenceSpec is a Pydantic model holding one or more wavelengths (SI meters), angles (θ, ϕ) in degrees, and a Jones vector. A convenience constructor at_wavelength_nm(wl_nm) wraps the typical normal incidence single-wavelength case.
- MaterialIndex is a Pydantic record of $(n, k, \lambda, \text{valid_range})$ plus a complex_index property.
- DesignState is a mutable dataclass scratchpad used by notebooks and legacy GUI code; the LLM-driven workflow does not use it. Its presence is noted here to avoid confusion.

5.2 Configuration Layer

configs/settings.py defines RCWAConfig (a BaseSettings with env_prefix="RCWA_" and defaults harmonics=3, resolution=64, minibatch_size=20), and a plain-Python PathsConfig that derives project_root, output_dir, papers_dir, and material_dir from the location of the settings file. It auto-creates outputs/ on import and sets TF_CPP_MIN_LOG_LEVEL=3 to silence TensorFlow.

The settings singleton is imported by MetaboxBackend, physics/materials.py, and ExportGDSTool - the three places that need absolute paths or RCWA knobs.

5.3 Physics Layer

5.3.1 Backend ABC

physics/backends/base.py defines SimulationBackend with three abstract methods: simulate_unit_cell, sweep_unit_cells, and propagate_field. The ABC docstring enumerates the two *hard rules* every backend must obey:

1. Safe to construct without heavy dependencies installed.

2. No TF/torch tensors leak upward; everything is converted to numpy/Python builtins before return.

A paired BackendRegistry resolves backend names, including an "auto" mode that tries "metabox" → "s4" → "analytical" and returns the first available.

5.3.2 The three backends

Table 5.2 summarises the three concrete backends. The analytical backend is a numpy-only heuristic port of the old core/automl.py inner loop. The metabox backend is the full RCWA path. S4 is a stub whose only functional method is is_available().

Table 5.2: Simulation backends.

Backend	Deps	Supports	Role	Status
AnalyticalBackend	numpy only	unit cell, diameter sweep, Fraunhofer propagation	Fallback, CI, notebook iteration	[IMPLEM
MetaboxBackend	TF, metabox3	unit cell, diameter sweep, angular-spectrum propagation	Primary validation path	[IMPLEM
S4Backend	S4	–		[PARTIA

5.3.3 High-level facades

physics/rcwa.py::sweep_diameter is a one-line helper that builds an IncidenceSpec at the design wavelength, generates a linearly spaced diameter vector between params.min_diameter_nm and params.max_diameter_nm (using numpy.linspace(..., n)), and dispatches to the registry. physics/propagation.py::propagate performs the SI-unit conversion ($nm \to m$, $\mu m \to m$) and delegates to the chosen backend's propagate_field.

physics/materials.py is the *only* place outside backends/metabox.py that lazy-imports from vendor.metabox3 (Material class). It exposes list_materials() and get_material_index(name, wl_nm) with cubic CSV interpolation.

physics/fdtd.py is a static reference library of phase-vs-diameter curves at five wavelengths (405/532/633/850/1064 nm); no FDTD solver is called - these are tabulated numbers transcribed from published papers. An Excel ingest helper allows users to load their own (dimension, phase) tables.

5.4 Phase Layer

phase/profiles.py implements the three canonical phase masks described in §2: Fresnel zone lens, axicon, and spiral phase plate, plus a from_image(filepath) loader that maps $[0, 255]$ grayscale to $[0, 2\pi)$. Pure numpy; depends only on PIL and numpy.

phase/quantization.py implements uniform-interval quantisation into N levels on $[\varphi_{\min}, \varphi_{\max}]$, returning both an integer label grid and the bin-centre list.

5.5 Analysis Layer

Six single-file modules implement the Fourier-optics analysis stack:

- psf.py - Fraunhofer PSF with $4\times$ pad and scalable image-plane coordinates.
- mtf.py - MTF via FFT of PSF, radial average, diffraction-limit reference based on the formula $(2/\pi)[\arccos(f/f_c) - (f/f_c)\sqrt{1-(f/f_c)^2}]$.
- strehl.py - Strehl ratio via two FFTs over the same aperture, as in Eq. (2.8).
- zernike.py - Noll-indexed polynomials up to $j = 15$ with the exact radial polynomial recursion and dictionary of names (ZERNIKE_NAMES).
- encircled_energy.py - cumulative-ring summation and the "radius enclosing fraction x" helper.
- farfield.py - Fraunhofer propagation to a specific propagation distance, with physically-scaled (x, y) axes.

5.6 Optimization Layer

optimization/base.py defines Optimizer with a single abstract method optimize(params, *, target_fom, max_iters, **kwargs). Three concrete subclasses exist:

- [IMPLEMENTED] HeuristicOptimizer - computes a strehl estimate $\text{amp}^2 \cdot \min(1, \text{coverage})$ from a pre-computed RCWASweepResult; does not actually search the design space. Its history list is the same scalar repeated up to max_iters, which is a known limitation (see §15).
- [IMPLEMENTED] GradientOptimizer - the only component in the repository that performs true inverse design. It constructs a metabox3 Metasurface with set_structures_variable=True, wraps it in a LensAssembly, and runs assembly.optimize_single_lens_assembly with FigureOfMerit. Guarded by two independent try/excepts: one for TF import, one for metabox3 API drift (AttributeError caught and downgraded to a zero-FOM report). Called *outside* the LLM agent.
- [PARTIAL] SurrogateOptimizer - a documented skeleton that returns FOMResult(value=0.0, metadata={"status":"not_implemented"}).

5.7 Fabrication Layer

fabrication/gds.py::GDSEngine converts a quantised phase mask plus a phase-vs-diameter sweep into a GDSII file via gdsfactory. Highlights:

- Lazy gdsfactory import.
- Supports "Cylinder", "Cross" (12-point polygon), "Fin" (rotated 4-vertex rectangle) shapes.
- Rotational caching: one component per unique rounded diameter.
- Circular aperture clipping.
- Progress callback hook (used by notebooks).

The ExportGDSTool in src/metaopticsai/tools/gds_tools.py hard-anchors the output path to settings.paths.output_dir and refuses any output_name that would escape that directory.

5.8 Scientific Tool Layer (`src/metaopticsai/tools/`)

A richer, Pydantic-schema-driven tool API living above the physics core. Its intent is to serve as the catalog that a future MCP transport (or any direct caller) exposes to clients.

5.8.1 `BaseTool` and `ToolRegistry`

Each tool class declares class attributes name, description, Input (a Pydantic model) and optionally Output, plus the flag needs_store. The abstract _run(params) does the work; the base class provides both call(...) (async-aware) and call_sync(...) wrappers that validate inputs, invoke _run, and conditionally validate outputs.

An ArtifactStore Protocol declares the shape of the handle-based payload store the tools require but do not own: methods get, put, list. Concrete store implementations live outside the tools layer, injected at composition time.

5.8.2 The composition root

tools/__init__.py::build_tool_registry(store, backends, job_manager=None) assembles the full catalog. Lazy imports keep the plain from metaopticsai.tools import BaseTool cheap - no matplotlib/scipy/gdsfactory until a registry is actually built. Table 5.3 enumerates the tools wired up.

Table 5.3: Tools registered by `build_tool_registry`.

Tool name	Purpose
`get_fdtd_library`	List built-in FDTD reference entries.
`run_rcwa_sweep`	RCWA diameter sweep via a selected backend.
`list_materials`	Enumerate bundled CSV materials.
`get_material_index`	Interpolate $n + ik$ for one material at a wavelength.
`generate_phase_mask`	Produce an FZL, axicon, or SPP mask; return handle.
`analyze_psf`	Compute PSF + Strehl + FWHM; return handle.
`analyze_mtf`	Radial MTF + diffraction-limit reference; return handle.
`zernike_decompose`	Top-N Zernike coefficients.
`propagate_field`	Propagate phase mask through distance z.
`compute_max_intensity`	Max intensity of a stored PSF.
`compute_center_intensity`	On-axis intensity (differentiable FOM).
`compute_mtf_volume`	Volume under normalised MTF surface.
`export_gds`	Build GDSII from design + sweep handles.
`optimize_metalens`	Heuristic / gradient / surrogate optimisation.
`train_metamodel`	Submit a background metamodel-training job (requires `job_manager`).
`get_job`	Poll a submitted job.
`list_artifacts`	List stored artifact handles, optionally by kind.

5.8.3 Pydantic schemas

Every tool's input and (where applicable) output is a strict Pydantic model under tools/schemas/. See Appendix D for the field-by-field reference.

5.9 Utility Layer

Four small files provide:

- async_utils.py - to_async, maybe_await for mixing sync and async tool implementations.
- json_io.py - NumpyJSONEncoder and a tolerant safe_parse_llm_json that strips "‘json/"‘ fences before parsing.
- logging.py - idempotent root-logger setup; silences tensorflow, absl, h5py, matplotlib to ERROR.
- images.py - lazy matplotlib (Agg) helpers that render figures and 2-D arrays to base64-encoded PNG strings; used for sending images back as MCP ImageContent.

Chapter 6

LLM Architecture

6.1 Package Layout

The llm/ package is small by design:

- llm/__init__.py - a docstring stating that nothing in this package knows about physics; it only routes LLM tool calls to mcp_server.tools.dispatch.
- llm/groq_client.py - environment loading and Groq SDK construction.
- llm/agent.py - the system prompt, the tool-calling loop, and the AgentResult dataclass.
- llm/__main__.py - python -m llm "<prompt>" CLI.

6.2 Model and Configuration

The default model is llama-3.3-70b-versatile on Groq. Any Groq-hosted tool-calling model may be substituted via GROQ_MODEL. Table 6.1 lists the environment knobs read by get_model_config() and run_agent.

Table 6.1: Environment variables consulted by the LLM layer.

Variable	Default	Role
GROQ_API_KEY	– (required)	Groq credentials.
GROQ_MODEL	llama-3.3-70b-versatile	Model name.
GROQ_TEMPERATURE	0.1	Sampling temperature.
GROQ_MAX_TOKENS	1024	Max tokens per completion.
AGENT_MAX_ITERATIONS	5	Clamp on tool round-trips, hard max 5.

load_env() reads config/.env then <root>/.env with hand-parsed KEY=VALUE lines; it never overwrites variables already in os.environ. This avoids a dependency on python-dotenv and keeps precedence predictable.

get_client() raises a RuntimeError with a clear message if GROQ_API_KEY is unset. The groq SDK is imported locally to avoid a hard package-level dependency.

6.3 System Prompt

agent.py::SYSTEM_PROMPT is the single piece of "LLM engineering" in the entire codebase. Its structure is:

1. A role statement: "careful meta-optics design assistant".
2. A hard rule: never invent parameters - start from a template.
3. A seven-step procedure that maps 1-to-1 to the six MCP tools, with explicit conditionals: *if phase coverage* < 0.9 *or mean transmission* < 0.7*, call get_optimization_tips and adjust*.
4. A tool-call cap: at most five tool calls total, stop after one round of tips, report a short structured summary.

The prompt is deliberately prescriptive. It is not a reasoning scaffold; it is a procedural manual that compresses the five-stage workflow document (see mcp_server/workflow.py) into a reinforcement for the LLM.

6.4 Tool-Calling Loop

6.4.1 Entry point

run_agent(user_prompt, max_iterations=None) returns an AgentResult dataclass with four fields: final_text, iterations, tool_calls (a log), and stopped_on (either "final_text" or "iteration_cap").

6.4.2 Loop body

The loop implementation is simple and complete in a single file. Each iteration:

1. Call client.chat.completions.create with the full message history, the TOOLS descriptor list, and tool_choice="auto".
2. Extract choices[0].message and any tool_calls.
3. Append the assistant turn (preserving tool_calls IDs and the raw function.arguments string).
4. If no tool calls, return final_text.
5. Otherwise, for each tool call: JSON-decode arguments (bad JSON becomes {"ok":false,"error":".. dispatch via mcp_server.tools.dispatch, strip any "_full" key for the LLM-visible view, truncate the JSON string to 6000 characters, and append a role="tool" message.

6.4.3 What the LLM sees

Two mechanisms shrink the LLM's view of large results:

- _strip_full_arrays(result) removes the private "_full" key before the result is JSON-encoded for the model. run_simulation uses this to keep the raw sweep arrays on the Python side while giving the LLM only the summary metrics plus a 5-sample head.
- A hard [:6000] slice on the JSON string ensures no runaway tool output can blow the context window.

6.5 AgentResult

Every call records a complete trace:

- final_text - the model's last textual response.
- iterations - the number of loop passes.
- tool_calls - a list of {iteration, name, args, result} dicts; the full result (including any stripped _full payload) is preserved for the caller even though the LLM only saw the trimmed version.
- stopped_on - "final_text" if the loop exited naturally, "iteration_cap" if it ran out of budget.

llm/__main__.py serialises each tool call's args and result to 400-character excerpts and prints them when -show-tools is set.

6.6 LLM Architecture Diagram

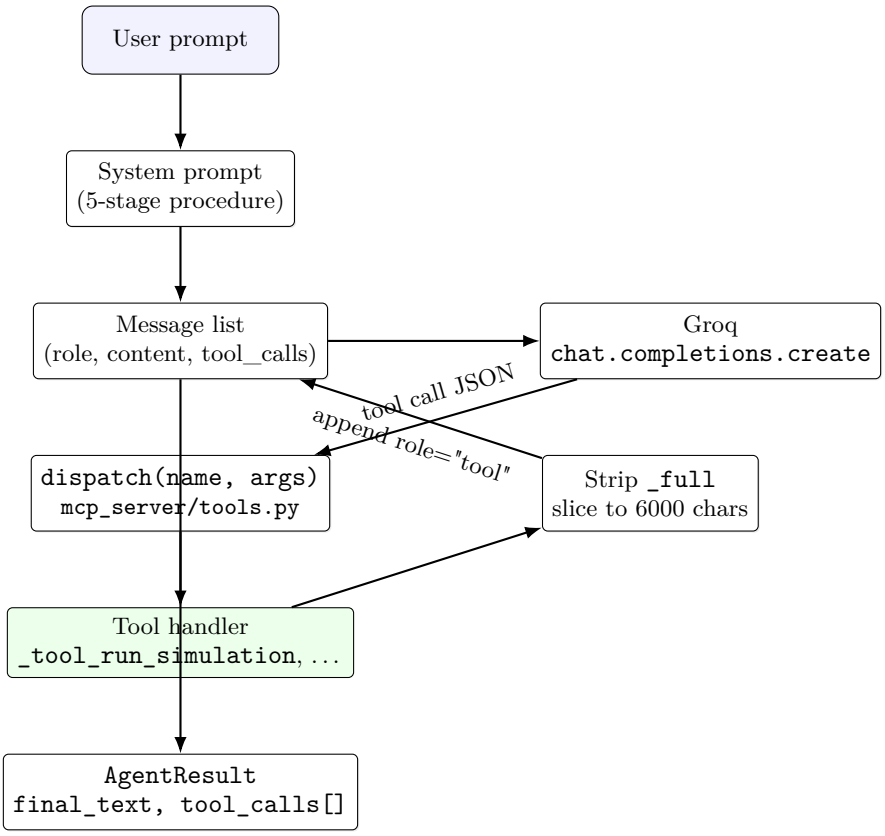


Figure 6.1: The LLM loop as actually implemented in llm/agent.py. The LLM sees the trimmed view; the AgentResult returned to the caller preserves the full one.

6.7 What the LLM Layer Does Not Do

To forestall interpretation errors:

- [FUTURE] There is no streaming of model output; each turn is a blocking create().

- [FUTURE] There is no RAG, vector search, or external memory. All "memory" is the message list within a single run_agent call.
- [FUTURE] There is no planner/critic split; the single assistant decides the next tool call entirely from the system prompt plus the running conversation.
- [FUTURE] There is no retry-on-failure except via the LLM itself re-reading an "ok": false tool result and choosing to call again.
- [FUTURE] No usage/cost reporting is captured.

Chapter 7

MCP Architecture

7.1 Positioning

The mcp_server/ package is *MCP-shaped* rather than *MCP-hosting*. Its tool descriptors are JSON-Schema objects in the shape consumed by both the Groq/OpenAI function-calling API and by the Model Context Protocol [1], but mcp_server.tools.dispatch(name, args) is invoked in-process from llm/agent.py; there is no stdio server. The mcp_server/README.md is explicit:

> "Single-agent, in-process. No multi-agent orchestration, no separate stdio process. Tools are called via mcp_server.tools.dispatch - the same JSON-Schema shape the MCP protocol expects, so wiring in a real MCP server later is a ∼30-line change."

This positioning is deliberate: the design keeps the tool catalogue small and the schema shape future-compatible, while sidestepping process-management complexity for a research prototype.

7.2 Package Layout

- mcp_server/__init__.py - a small side-effectful loader that inserts src/ into sys.path if the metaopticsai package is not already importable. This lets python examples/demo.py work without a preceding pip install -e ..
- mcp_server/tools.py - the six tool implementations, the _HANDLERS dispatch map, the TOOLS JSON-Schema descriptor list, and dispatch().
- mcp_server/templates.py - the TEMPLATES dict with three verified starting points (TiO_2@532 nm, Si@850 nm, GaN@405 nm).
- mcp_server/workflow.py - the canonical five-stage workflow guide as a plain string.
- mcp_server/simulator.py - a thin wrapper over metaopticsai.physics.rcwa.sweep_diameter that Pydantic-validates inputs and emits JSON-safe dicts.

7.3 The Six Verified Tools

Table 7.1 enumerates the six MCP-shaped tools exposed to the LLM. Appendix B contains the full input/output schema and implementation notes for each.

Table 7.1: Verified MCP-shaped tool catalogue (mcp_server/tools.py).

Tool	Stage	Purpose
list_templates	1	Enumerate verified starting points.
get_template	1	Full parameter dict for one template.
get_workflow_guide	–	Return canonical 5-stage workflow text.
validate_design	3	Pydantic-validate a candidate design and compute derived metrics (NA, $f/\#$, aspect ratio, mean diameter).
run_simulation	4	RCWA diameter sweep (metabox3 or analytical fallback).
get_optimization_tips	5	Rule-based textual suggestions from a sim result.

7.4 Determinism and Error Handling

Two design decisions distinguish this toolbox from a typical LLM-callable Python surface:

- **Never raise past the boundary.** The module docstring is explicit: a bad call returns {"ok": false, "error": "..."} so the LLM can read the message and retry. Each handler is wrapped in two layers: an inner try/except (ValidationError, KeyError, ValueError, RuntimeError) for expected error classes, and an outer catch-all that includes a trimmed traceback for debugging.
- **Numeric outputs are backend-only.** The sweeper runs the full result, then strips the large arrays (sweep_values_nm, amplitudes, phases_rad) into a 5-sample sweep_head preview before returning. The full arrays remain in "_full", which llm/agent.py filters out before sending to the model. This keeps the LLM from either reasoning over or regurgitating hundreds of floats.

7.5 Templates as Ground Truth

mcp_server/templates.py::TEMPLATES is the entire corpus of "verified knowledge" the system has about meta-atom geometries. Each entry provides:

- A human-readable summary.
- wavelength_nm, material, shape.
- A notes field citing the published reference the parameters are derived from (e.g. [5]).
- A params dict that is guaranteed to round-trip through MetalensDesignParams(**template["params"]

Adding a template is a one-dict append; Pydantic validation at the first call blocks any malformed entry from reaching the LLM. This is the mechanism that forbids the agent from "inventing" unit-cell geometries: it is permitted to *tweak* the dict, not to supply a fresh one from imagination.

7.6 The Workflow Guide

mcp_server/workflow.py::WORKFLOW_GUIDE is the longest piece of plain text in the mcp_server/ package. Its structure:

1. Pick a starting template.

2. Adapt parameters (wavelength, focal length, diameter, material, height, min/max diameter, periodicity, with pithy ranges and a cross-field constraint reminder: "periodicity > max_diameter + 20 nm").
3. Validate. Always call validate_design before physics.
4. Simulate. Call run_simulation; success criterion is phase_coverage >= 0.9 AND mean_transmissio >= 0.7.
5. Interpret and iterate. Below target ⇒ get_optimization_tips. Cap at 3-5 round-trips.

This is a *manual* the LLM can call up at any point if the system prompt alone is insufficient context. The tool itself is get_workflow_guide and takes no arguments.

7.7 The Simulator Wrapper

mcp_server/simulator.py::run_diameter_sweep(params, n_samples, backend) does three things:

1. Round-trip params through MetalensDesignParams (second validation after the LLM's hopefully-val payload).
2. Call metaopticsai.physics.rcwa.sweep_diameter(...).
3. Return a dict with every numpy.ndarray converted to a list so the result is JSON-serialisable.

A separate available_backends() introspection helper surfaces which backends can actually run (used by the demo header).

7.8 MCP Architecture Diagram

7.9 Extension Model

Two documented extension points:

- **Add a template.** Append a dict to TEMPLATES in templates.py. Pydantic validation at first use blocks malformed entries.
- **Add a tool.** Implement _tool_yourthing(args) -> {"ok": ..., ...} in tools.py, append its JSON-Schema descriptor to TOOLS, and register it in _HANDLERS. No other file needs to change.

The in-process design means an added tool is live on the next run_agent call without a restart.

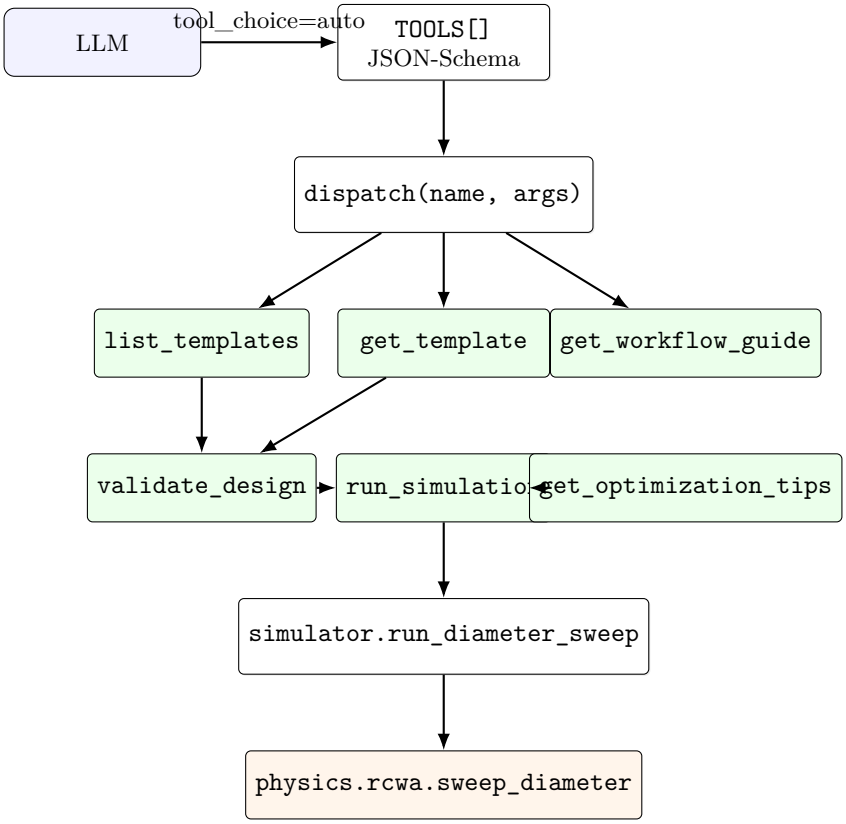


Figure 7.1: The MCP-shaped tool catalogue. Solid arrows denote dispatch; vertical adjacency reflects the staged dependency implied by the system prompt and workflow guide.

Chapter 8

The AI-Driven Workflow

8.1 The Canonical Five-Stage Procedure

The repository encodes a single, prescribed procedure for the LLM to follow. It is stated three times in the sources (once as a Markdown document in workflow.py, once as a system prompt in agent.py, and once as a user-facing table in both the top-level README.md and the mcp_server/README.md). All three agree on these stages:

1. **Pick a template.** list_templates → get_template(name). Templates are pre-validated and literature-derived; the LLM is forbidden from inventing new unit-cell geometries.
2. **Adapt.** The LLM edits a copy of the template's params dict to match the user's wavelength, aperture, focal length, and material.
3. **Validate.** validate_design(params={...}). Returns {valid, errors, warnings, derived}. The LLM is explicitly told to fix all errors before continuing.
4. **Simulate.** run_simulation(params={...}, n_samples=20). Returns a backend name, phase coverage, mean transmission, elapsed time, and a 5-sample head of the sweep.
5. **Interpret.** If phase coverage < 0.9 or mean transmission < 0.7, call get_optimization_tips(resul apply, and re-simulate at most once. Then stop and write a short structured summary.

Figure 8.1 depicts the stage graph including the conditional retry edge.

8.2 Walkthrough: What `run_agent` Actually Does

A step-by-step trace of run_agent(user_prompt):

1. load_env() reads config/.env and the root .env.
2. AGENT_MAX_ITERATIONS is read and clamped into $[1, 5]$.
3. get_client() builds a Groq client (raises on missing key).
4. get_model_config() returns the triple {model, temperature, max_tokens}.
5. The initial messages list is [{system}, {user}].
6. The loop runs up to 5 iterations, each doing one chat.completions.create with tools=TOOLS, tool_choice="auto".

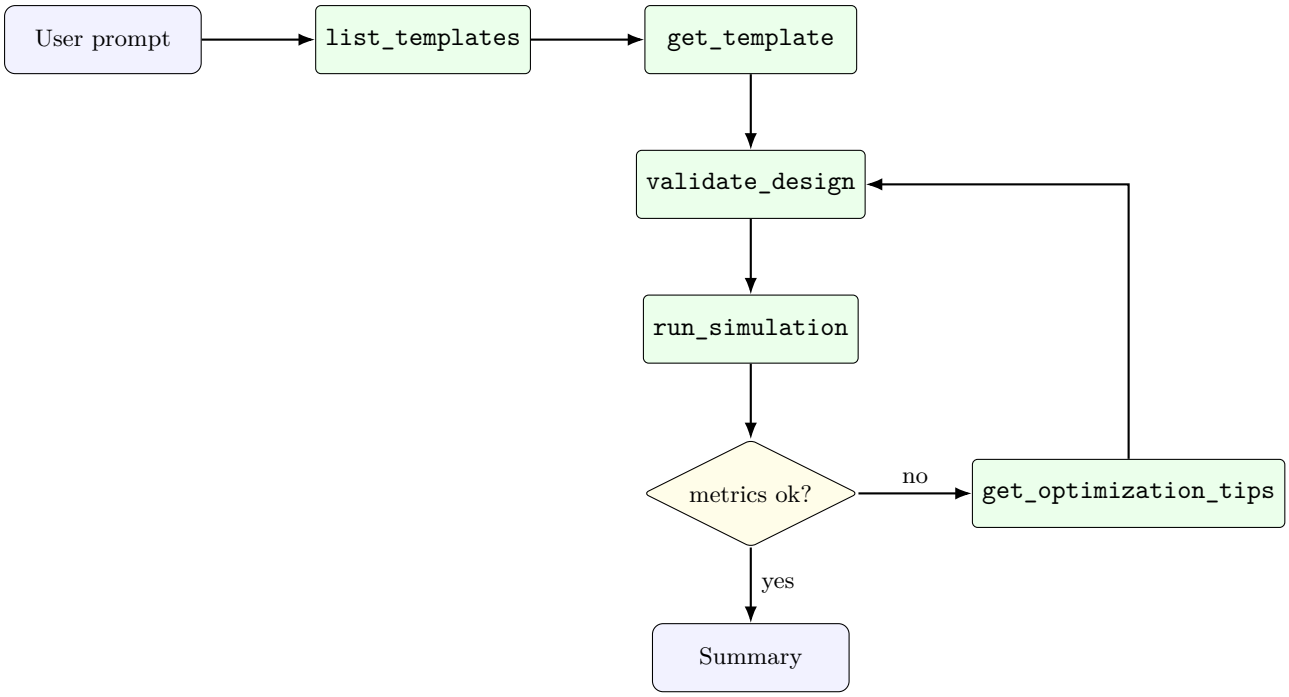


Figure 8.1: The five-stage design workflow with the single conditional retry edge. The iteration cap stops the loop at five tool calls regardless.

7. Every tool_call is logged in tool_call_log, dispatched through mcp_server.tools.dispatch, and its result (minus "_full") is appended as a role="tool" message, truncated to 6000 chars.
8. If the model returns a message with no tool calls, that message's content becomes final_text and the loop exits.
9. Otherwise, on falling out at iteration 5, the function returns AgentResult(final_text="[Agent hit iteration cap...]", stopped_on="iteration_cap").

8.3 What the User Sees

The user receives:

- A short textual summary of the final design parameters and the achieved metrics (AgentResult.fi
- When invoked via python -m llm "..." -show-tools or examples/demo.py, an itemised tool-call trace with the arguments and (truncated) result of each call.
- A one-line footer: (iterations=..., stopped_on=..., tool_calls=...).

8.4 What the LLM Can and Cannot Do

By construction:

- The LLM can call tools in any order; nothing enforces the five-stage ordering at the dispatch level. The ordering is only reinforced by the system prompt and workflow guide. A model that ignores the prompt and calls run_simulation first will receive a ValidationError-formatted "ok": false result and almost certainly correct itself.

- The LLM cannot inject numeric physics outputs into its response; the only numbers it *can* report are those it read back from a tool result. There is no post-hoc verification step, so adversarial prompts could still cause the LLM to misquote numbers. This is a limitation of the design, documented in §15.
- The LLM cannot export GDSII, run gradient optimisation, or propagate a field: those tools live in the src/metaopticsai/tools/ layer, not in the six exposed through TOOLS. Exposing them would require an additional handler in mcp_server/tools.py.

8.5 Dry-Run Mode

examples/demo.py -no-llm exercises the pipeline with no LLM in the loop. It calls dispatch five times in the canonical order (list_templates, get_template(name="tio2_metalens_532nm"), validate_design, run_simulation(n_samples=20), get_optimization_tips) and prints a compact summary. This mode is used both as a smoke test and as the fallback when GROQ_API_KEY is unset:

```
if not os.environ.get("GROQ_API_KEY"):
    print("GROQ_API_KEY not set -- falling back to --no-llm dry run.")
    print("(Set it in config/.env or the repo-root .env to run live.)\n")
    return dry_run()
return live_run(args.prompt, show_tools=not args.quiet)
```

Listing 8.1: Fallback branch in examples/demo.py.

Chapter 9

Meta-Optics Computational Framework

9.1 Overview

Beneath the LLM and MCP layers sits a conventional scientific-computing stack. This chapter documents the computational framework as it is implemented, with specific attention to what each backend actually computes and the SI-unit boundary at which numeric arrays enter and leave.

9.2 The Simulation Backend ABC

A SimulationBackend must implement three operations:

simulate_unit_cell(params, incidence) Return a UnitCellResult for a single meta-atom.

sweep_unit_cells(params, incidence, sweep_param, sweep_values) Return a RCWASweepResult for an array of parameter values along a named axis (currently only sweep_param="diameter").

propagate_field(phase_mask, wavelength_m, pixel_size_m, distance_m, refractive_index) Return a FieldResult with a 2-D intensity array, peak intensity, shape, wavelength, and backend name.

is_available() returns True iff all heavy dependencies and resources are present, which is why the three backends can coexist in one registry without import ordering trouble.

9.3 AnalyticalBackend

physics/backends/analytical.py is a numpy-only heuristic. Its model is intentionally simple:

Local phase. Equation (2.2) expresses the pillar phase as the leading-order thin-film accumulated phase $(n_p - 1)h/\lambda \cdot 2\pi$ scaled by a logistic "realisation factor" $\sigma((d - 50)/d)$. This yields a smoothly varying, monotone phase versus diameter curve with gradient-friendly shape.

Local amplitude. A product of two penalty factors discourages pathological geometries:

$$|t|(d) = 0.7 + 0.3 \cdot \text{ar_factor}(d, h) \cdot \text{fill_factor}(d, p), \tag{9.1}$$

where ar_factor punishes aspect ratios above 10 and fill_factor punishes fill ratios above 0.8.

Material table. A fixed 4-entry _N_MATERIAL = {"TiO2": 2.35, "Si": 3.48, "GaN": 2.32, "SiN": 2.0} is used; this is explicitly a stand-in for a wavelength- dependent index. For dispersive studies the MetaboxBackend path with Material's CSV-based $n(\lambda)$ is required.

Propagation. Fraunhofer (zero-padded 2-D FFT), returning a normalised intensity. The pixel-to-image-plane coordinate scaling is not computed by this backend; see analysis/psf.py and analysis/farfield.py for the physically-scaled form.

The analytical backend is the fallback: when TF is unavailable, the agent's run_simulation silently uses it, and the simulator reports backend="analytical" in the LLM-visible result so the agent (and the user) can see which path was taken.

9.4 MetaboxBackend

physics/backends/metabox.py is the high-fidelity path.

9.4.1 Unit-cell construction

_build_protocell(params, min_diameter_nm, max_diameter_nm) constructs a metabox3 ProtoUnitCell whose single layer contains a Circle of material index n_p and radius equal to a Feature spanning the sweep interval $[d_{\min}, d_{\max}] \cdot 10^{-9}$ metres with 20 samples. The pillar sits in air ($n_r = 1.0$) above a substrate of tran_index=params.substrate_index. The unit cell is square with pitch $p =$ periodicity_nm$\cdot 10^{-9}$.

9.4.2 Simulation configuration

_build_simconfig builds a SimConfig with xy_harmonics from the design parameters, resolution from settings.rcwa.resolution (default 64), and a mini-batch size of min(n_samples, settings.rcwa.mi
The flags return_tensor=True, return_zeroth_order=True, use_transmission=True, include_z_comp=False fix the output tensor shape to (n_wl, n_samples, 2 polarisations) containing the zeroth-order t_x, t_y transmission amplitudes.

9.4.3 Sweep

sweep_unit_cells converts the input sweep_values (nanometres) to metres, builds a 1-D parameter tensor of shape (1, N), calls rcwa.simulate_parameterized_unit_cells(param_tensor, proto, inc, cfg), and extracts t_x. The phase array is np.unwrap(np.angle(tx)) with the first element subtracted so the sweep starts at 0. Phase coverage is $(\max\varphi - \min\varphi)/(2\pi)$, mean transmission is $\overline{|t_x|}$.

9.4.4 Propagation

propagate_field wraps the phase mask into a Field2D, builds an angular-spectrum transfer function via propagation.get_transfer_function(ref_idx, prop_dist), and calls propagation.propagate(

H). The returned intensity is extracted via .get_intensity().numpy()[0,0] and normalised to peak unity.

9.4.5 Guarded availability

is_available() performs a one-shot test: it imports TensorFlow and metaopticsai.vendor.metabox3 inside a try/except, logs on failure (not a stack trace), caches the result, and returns. Any method that reaches the physics call also runs _require(), which raises a clean RuntimeError with install guidance.

9.5 S4Backend (Stub)

physics/backends/s4.py declares the S4 name, implements is_available() with a plain import S4, and raises NotImplementedError on each of the three simulation methods. Its docstring sketches how to populate the methods and marks a "validation roadmap" note. The backend is registered in the default registry so that backend="s4" is a selectable value for the LLM-visible tool, but calling it in the current state produces an "ok":false error with the stub message - which the LLM then relays back to the user.

9.6 High-Level Facades

Two facades expose the backend set to the rest of the codebase:

- physics.rcwa.sweep_diameter(params, *, wavelength_nm, n, backend) - used by both mcp_server/sim and src/metaopticsai/tools/rcwa_tools.py.
- physics.propagation.propagate(phase_mask, *, wavelength_nm, pixel_size_um, distance_um, refractive_index, backend) - used by src/metaopticsai/tools/propagation_tools.py.

Each facade is the single hop at which natural units (nm, μm) are converted to SI.

9.7 Material Index Lookup

physics/materials.py::get_material_index(name, wavelength_nm) is the only path by which *wavelength-dependent* refractive indices enter the computation. It calls vendor.metabox3.rcwa.Materi custom_csv_dir= settings.paths.material_dir) and reads the complex index at the requested wavelength. A list_materials() enumerates every CSV in that directory; 12 materials ship with the vendored package.

Within the primary mcp_server/ toolbox, this facade is *not* consumed - MetaboxBackend uses the fixed _DEFAULT_N dict for the four approved materials. The material-index facade is used by the get_material_index tool in src/metaopticsai/tools/materials_tools.py, which is part of the broader [CONFIGURED] catalogue.

9.8 Scientific Computation Pipeline Diagram

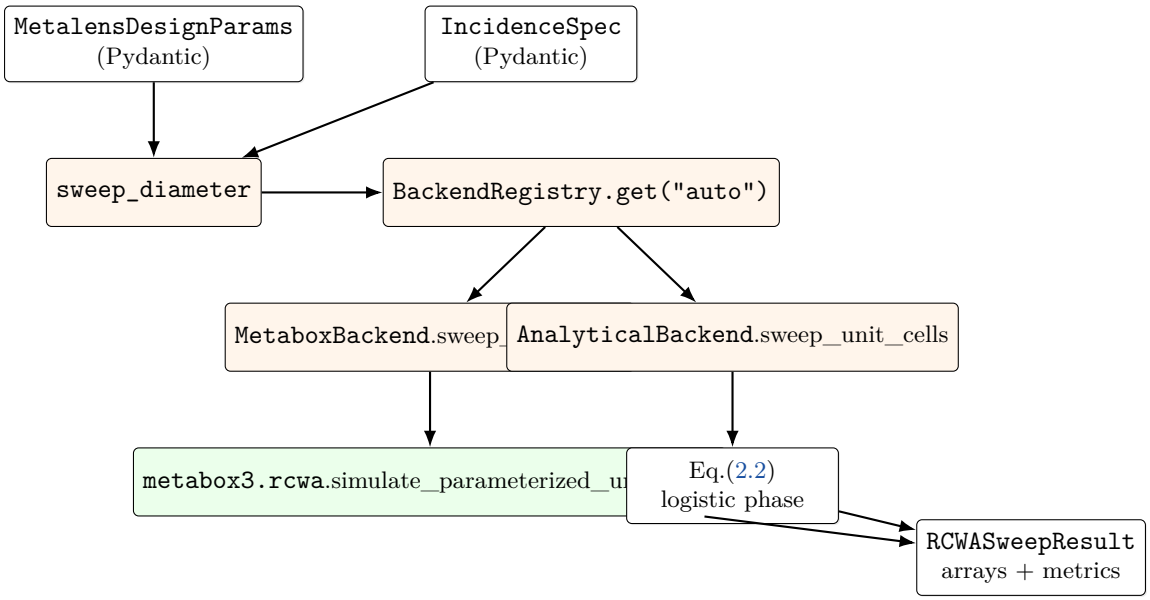


Figure 9.1: The scientific computation pipeline for a diameter sweep. The registry's `get("auto")` chooses metabox when available and analytical otherwise.

Chapter 10

Design and Simulation Pipeline

10.1 Pipeline Boundaries

The design pipeline inside MetaCode is best described by the data that crosses each boundary. Table 10.1 tracks every boundary and the shape of the data at it.

Table 10.1: Pipeline boundaries and data shapes.

Boundary	Direction	Shape at boundary
LLM ↔ MCP tool	both	JSON object; Pydantic-validatable.
MCP tool ↔ physics facade	both	Pydantic `MetalensDesignParams` + `IncidenceSpec`.
Facade ↔ backend	both	Same Pydantic objects + numpy array.
Backend ↔ `metabox3`	both	TF tensors (*inside* the adapter only).
Backend ↔ caller	return	`RCWASweepResult`/`UnitCellResult`/`FieldResult` dataclass with numpy arrays.
Scientific tool ↔ artifact store	both	12-hex UUID handle + payload.
Fabrication ↔ disk	write	`.gds` path, size, elapsed.

10.2 Stages in Depth

10.2.1 Stage 1 – Template selection

The tool list_templates returns a flattened compact list containing only the fields the LLM needs to pick:

```
{
  "ok": true,
  "templates": [
    {"name": "tio2_metalens_532nm", "summary": "...", "wavelength_nm": 532.0,
     "material": "TiO2", "shape": "cylinder"},
    {"name": "si_nir_metalens_850nm", ...},
    {"name": "gan_uv_metalens_405nm", ...}
  ]
}
```

Listing 10.1: Shape of `list_templates` output.

The full payload with the parameter dict is only served by get_template(name=...) on request, so the compact index is cheap to show to the LLM for routing decisions.

10.2.2 Stage 2 – Parameter adaptation

Adaptation is *not* a tool. The LLM edits the dict in its own message, then passes it as params={...} to validate_design. The adaptation step is therefore the single place where the LLM is trusted to produce numbers; the next tool enforces that those numbers lie in the admissible region.

10.2.3 Stage 3 – Validation

validate_design(params={...}) does three things:

1. Instantiates MetalensDesignParams(**params). All per-field range constraints and the cross-field _check_geometry validator run here.
2. If validation fails, returns a list of {field, msg} errors.
3. If validation succeeds, computes two soft warnings (aspect_ratio>10, numerical_aperture>0.85) and reports the derived quantities: NA, f-number, aspect ratio, mean diameter.

The returned object always carries "ok": true; the pass/fail is in the top-level "valid" key. This distinction matters: tool-level errors (bad JSON, unknown template) use "ok": false; scientific-level failures (invalid geometry) use "valid": false. The two live in orthogonal keys so the LLM can route them separately.

10.2.4 Stage 4 – Simulation

run_simulation(params={...}, n_samples=20, backend="auto") hands the dict to run_diameter_sweep. The backend returns a result with the full numpy arrays; the tool *keeps the full result* in the private "_full" key but exposes only a 5-sample head and the scalar metrics to the LLM:

```
{
  "ok": true,
  "result": {
    "backend": "metabox",
    "sweep_param": "diameter",
    "wavelength_nm": 532.0,
    "phase_coverage": 0.97,
    "mean_transmission": 0.88,
    "elapsed_s": 1.42,
    "n_samples": 20,
    "sweep_head": {
      "sweep_values_nm": [80.0, 91.6, 103.2, 114.7, 126.3],
      "phases_rad": [0.0, 0.42, 0.95, 1.73, 2.56],
      "amplitudes": [0.91, 0.93, 0.94, 0.94, 0.92]
    }
  }
}
```

Listing 10.2: LLM-visible shape of run_simulation.

The private "_full" key (not shown) holds the full 20-element vectors. llm/agent.py::_strip_full drops it before encoding the result for the model.

10.2.5 Stage 5 – Interpretation

get_optimization_tips(result={...}) applies three rule-based thresholds to the summary metrics in the previous result:

- **Coverage** $< 0.9 \Rightarrow$ "widen the diameter sweep range or increase pillar_height_nm".
- **Mean transmission** $< 0.7 \Rightarrow$ "reduce pillar_height_nm or cap max_diameter_nm at ∼(periodicity_nm - 40)".
- **Coverage** ≥ 0.95 **and transmission** $\geq 0.85 \Rightarrow$ "design is ready for phase-mask synthesis and GDS export".
- Otherwise $\Rightarrow$ "try n_samples=30-40 for a smoother phase curve".

This is intentionally a crisp decision table so the LLM can trust and relay the advice verbatim.

10.3 Beyond Simulation: What the Agent Does Not Do

The six tools *stop* at Stage 5. Not exposed to the LLM:

- [CONFIGURED] Phase-mask generation (generate_phase_mask in src/metaopticsai/tools/phase_tools.py Produces FZL/axicon/SPP masks and stores them by handle.
- [CONFIGURED] PSF/MTF/Zernike analysis (analyze_psf, analyze_mtf, zernike_decompose). Pure numpy, no backend dependency.
- [CONFIGURED] Field propagation (propagate_field). Uses the registry's preferred backend; converts units at the facade.
- [CONFIGURED] GDS export (export_gds). Combines an FZL phase mask with a sweep's phase-to-diame interpolation and writes a GDSII file under settings.paths.output_dir.
- [CONFIGURED] Optimisation (optimize_metalens, train_metamodel). Backed by HeuristicOptimizer, GradientOptimizer, and ([PARTIAL]) SurrogateOptimizer.

These tools exist in the second (src/metaopticsai/tools/) layer with full Pydantic schemas, but no transport in the shipping repository binds them to the LLM. Exposing them is a matter of adding thin handler shells in mcp_server/tools.py (or hosting a real MCP server over build_tool_registry).

10.4 A Scientist-Style Pipeline

If the broader tool catalogue is driven directly (e.g. from a notebook), the pipeline a scientist would run looks like Figure 10.1. Every step produces a Artifact handle; the next step consumes handles rather than raw payloads, which keeps the API uniform across scientific object types.

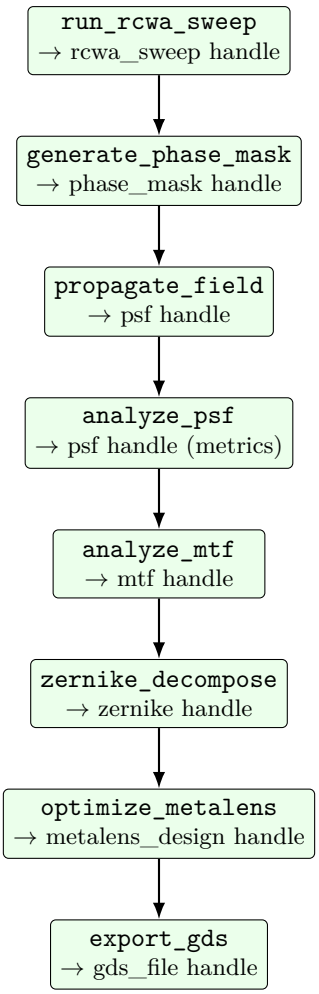


Figure 10.1: A plausible scientist-driven pipeline using the broader tool catalogue. Each node writes an artifact handle; edges carry the handle, not the payload. No current LLM transport drives this pipeline ([CONFIGURED]).

Chapter 11

End-to-End Case Studies

The two case studies in this chapter trace the agent's behaviour on realistic prompts *as the code would run them*. Numeric values are grounded in mcp_server/templates.py; agent-side wording is representative, not deterministic (temperature 0.1, not 0).

11.1 Case Study 1 – A Green Visible Metalens

Prompt. "*Design a TiO_2 metalens for green light (532 nm). Start from a verified template, validate it, run a 20-diameter RCWA sweep, and report phase coverage and mean transmission.*" This is the DEFAULT_PROMPT in examples/demo.py.

Expected trace.

1. list_templates() returns the three-entry index. The agent selects tio2_metalens_532nm.
2. get_template(name="tio2_metalens_532nm") returns:

```
{
  "wavelength_nm": 532.0, "focal_length_mm": 2.0,
  "diameter_mm": 0.5, "pillar_material": "TiO2",
  "pillar_height_nm": 600.0,
  "min_diameter_nm": 80.0, "max_diameter_nm": 300.0,
  "periodicity_nm": 350.0
}
```

   The agent passes this verbatim (adaptation is a no-op for the default prompt).
3. validate_design(params={...}) returns valid=true, derived NA=0.124, f/#=4.0, aspect_ratio=3.16, no warnings.
4. run_simulation(params={...}, n_samples=20) with backend="auto" selects the first available backend. Representative outcomes:
   - backend="metabox" (TF installed): ∼1-2 s, phase coverage $\gtrsim 0.95$, mean transmission $\gtrsim 0.85$.
   - backend="analytical" (fallback): sub-second, heuristic values with coverage close to 1 and transmission around 0.9 (controlled by Eq. (2.2)).
5. Both metrics clear the thresholds; the LLM is instructed to stop after one round of tips or when metrics are acceptable. It calls get_optimization_tips(result= {...}) once, receives the "design is ready" branch, then writes a short summary.

Expected user-facing summary. A short paragraph naming the template, listing the final parameters, and reporting phase_coverage, mean_transmission, and the derived NA / f-number. No numeric physics claim appears in the summary that was not read from a tool result.

11.2 Case Study 2 – A UV Metalens with GaN

Prompt. *"Design a UV metalens for 405 nm using GaN pillars."*

Expected trace.

1. list_templates() returns the three entries. The agent selects gan_uv_metalens_405nm.
2. get_template returns:

```
{
  "wavelength_nm": 405.0, "focal_length_mm": 1.0,
  "diameter_mm": 0.3, "pillar_material": "GaN",
  "pillar_height_nm": 450.0,
  "min_diameter_nm": 40.0, "max_diameter_nm": 170.0,
  "periodicity_nm": 200.0
}
```

   Note max_diameter_nm=170 and periodicity_nm=200: the margin is exactly 30 nm, comfortably clear of the *20 nm wall* rule. The template is validated at ingestion time.

3. validate_design passes. NA = 0.149, aspect_ratio = 4.3. No warnings.
4. run_simulation is run. In the metabox path, the sweep is 20 points in $[40, 170]$ nm with harmonics $(3, 3)$. Representative metrics are coverage $\sim$0.9–0.95, mean transmission $\sim$0.8.
5. If coverage comes back < 0.9, the agent calls get_optimization_tips, which recommends increasing pillar_height_nm to accumulate more propagation phase (consistent with Eq. (2.2)). The agent may then raise pillar_height_nm to e.g. 500 nm and re-run.

Observation. The retry edge is the only control flow the LLM is given at the physics boundary. A well-prompted model that observes coverage $= 0.88$ will call get_optimization_tips, read the suggestion, update the dict, and re-simulate - without any Python code enforcing that sequence.

11.3 What Breaks the Workflow

Three realistic failure modes a user can produce without touching the code:

Impossible aperture. A prompt like "design a 50 mm metalens for 532 nm with focal_length_mm=0.1" pushes the agent toward numerical apertures above 0.995. validate_design will not reject such a value (the per-field constraints only bound ranges, not NA), but it will emit the "NA>0.85" warning. The physics backend may still accept the input and compute something, but the Strehl estimate will be poor; the agent should notice and surface the warning in its summary.

Over-filled unit cell. A prompt that nudges max_diameter_nm too close to periodicity_nm (less than the 20 nm margin) triggers _check_geometry and returns a valid=false payload with a specific error string. The system prompt instructs the LLM to "fix the reported errors and validate again", so a well-behaved agent corrects and continues.

Material drift. A prompt that mentions a material outside {"TiO2","Si","GaN","SiN"} produces a ValidationError on the pillar_material Literal. The LLM must substitute a valid material (typically picking the nearest template's material).

11.4 Reading the Trace

Both the demo and the module CLI print per-call traces. A reader auditing a run should check:

- Which backend produced each number (result.backend is "metabox" or "analytical"). Analytical results are heuristic; do not use them for publication-grade claims.
- Whether stopped_on is "final_text" or "iteration_cap". The latter indicates the agent ran out of budget and the final summary was synthesised by the loop itself, not the model.
- Whether any tool call returned "ok": false. Those messages are kept in the trace so a human can see what the LLM saw and decide whether its subsequent recovery was sound.

Chapter 12

Data and Control Flow

12.1 Data Classes Across the Stack

Table 12.1 lists the principal data objects and where they are constructed and consumed.

Table 12.1: Principal data objects, their type, and lifetime.

Object	Type	Where constructed	Where consumed
`MetalensDesignParams`	Pydantic	`validate_design`, `simulator.py`	every backend, every scientific tool
`IncidenceSpec`	Pydantic	`sweep_diameter`, `tools/rcwa_tools.py`	backends, `simulator.py`
`MaterialIndex`	Pydantic	`get_material_index`	`tools/material_tools.py`
`Artifact`	frozen dataclass	`ArtifactStore.put`	`ArtifactStore.get`, `Artifact.summary`
`UnitCellResult`	dataclass	`Simulation.unit_cell`	all backends, `simulate_unit_cell`
`RCWASweepResult`	dataclass	`Simulation.rcwa_sweep`	`rcwa_sweep_vs_diameter`, `optimization_tools.py`
`FieldResult`	dataclass	`Simulation.propagate`	`propagate_field`
`PSFResult`	dataclass	`analyze_psf` (internally)	`report`
`FOMResult`	dataclass	every `optimize_metalens` tool, caller of `Optimizer.optimize`	notebook plotting
`DesignState`	dataclass	notebook / legacy GUI code	GUI / demo output
`AgentResult`	dataclass	`run_agent`	CLI

12.2 Numeric Data Flow

The repository's determinism relies on a one-way flow: numeric arrays originate in a backend, are converted to JSON-safe lists at the simulator.py boundary, and are then trimmed to a

5-sample head before being handed to the LLM. Figure 12.1 depicts this with the trimming layers made explicit.

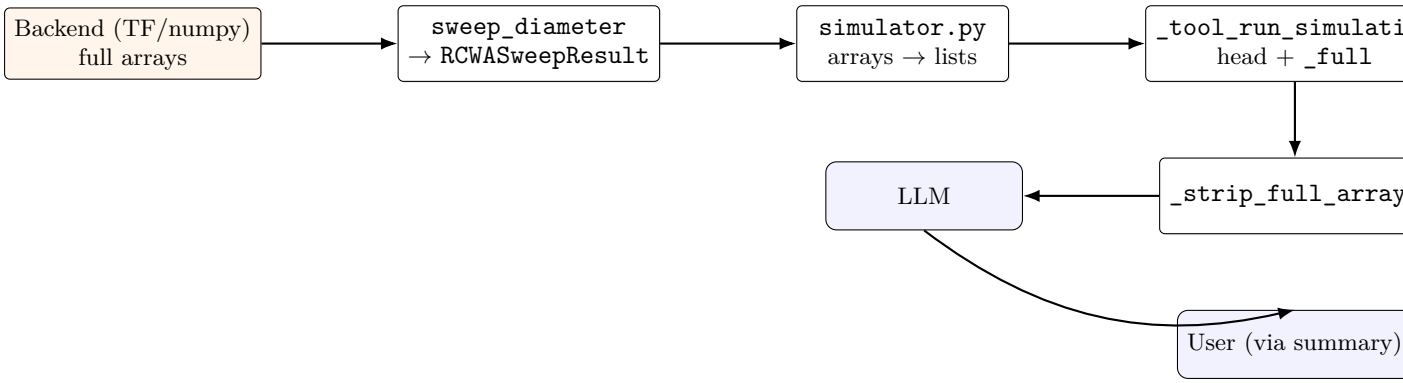


Figure 12.1: Numeric data flow with every size-reducing step shown. The LLM only ever observes the sweep_head + scalar summary, not the full sweep.

12.3 Control Flow in the Agent Loop

Procedurally, the agent is a tight loop:

```
for iteration in range(1, max_iterations + 1):
    resp = client.chat.completions.create(
        model=cfg["model"], temperature=cfg["temperature"],
        max_tokens=cfg["max_tokens"], tools=TOOLS,
        tool_choice="auto", messages=messages,
    )
    msg = resp.choices[0].message
    calls = _extract_tool_calls(msg)

    assistant_turn = {"role": "assistant", "content": msg.content or ""}
    if calls:
        assistant_turn["tool_calls"] = [...]
    messages.append(assistant_turn)

    if not calls:
        return AgentResult(final_text=msg.content or "", ...)

    for c in calls:
        name = c.function.name
        args = _parse_json(c.function.arguments)
        result = dispatch(name, args)
        tool_call_log.append({"iteration": iteration, "name": name,
                              "args": args, "result": result})
        messages.append({
            "role": "tool", "tool_call_id": c.id, "name": name,
            "content": json.dumps(_strip_full_arrays(result), default=str)[:6000],
        })
```

Listing 12.1: Agent loop body, condensed.

Two aspects are worth noting:

- *Tool calls accumulate within an iteration.* If the model emits more than one tool call in a single response, all are dispatched in order and each gets its own role="tool" message in the next request's history. The 5-iteration budget bounds *round-trips*, not raw calls.
- *Malformed JSON is not fatal.* json.JSONDecodeError becomes {"ok": false, "error": "Malformed JSON arguments..."}; the LLM reads the error and typically re-emits valid arguments on the next turn.

12.4 Error Flow

Three error "channels" exist and are distinct:

{"ok": false, "error": "..."} Tool-infrastructure errors: unknown tool name, JSON parse failure, backend RuntimeError, backend KeyError, or any catch- all with a trimmed traceback.

{"ok": true, "valid": false, "errors": [...]} Scientific validation errors from Pydantic. Only validate_design uses this shape.

AgentResult(stopped_on="iteration_cap") The LLM failed to produce a final text within the budget. The final summary is a sentinel string; the trace preserves every call attempted.

Within the scientific core, errors are typed Python exceptions (ValueError, NotImplementedError, RuntimeError, FileNotFoundError). The tool layer catches and rewrites them into one of the two JSON shapes above before reaching the LLM.

12.5 Dependency Graph

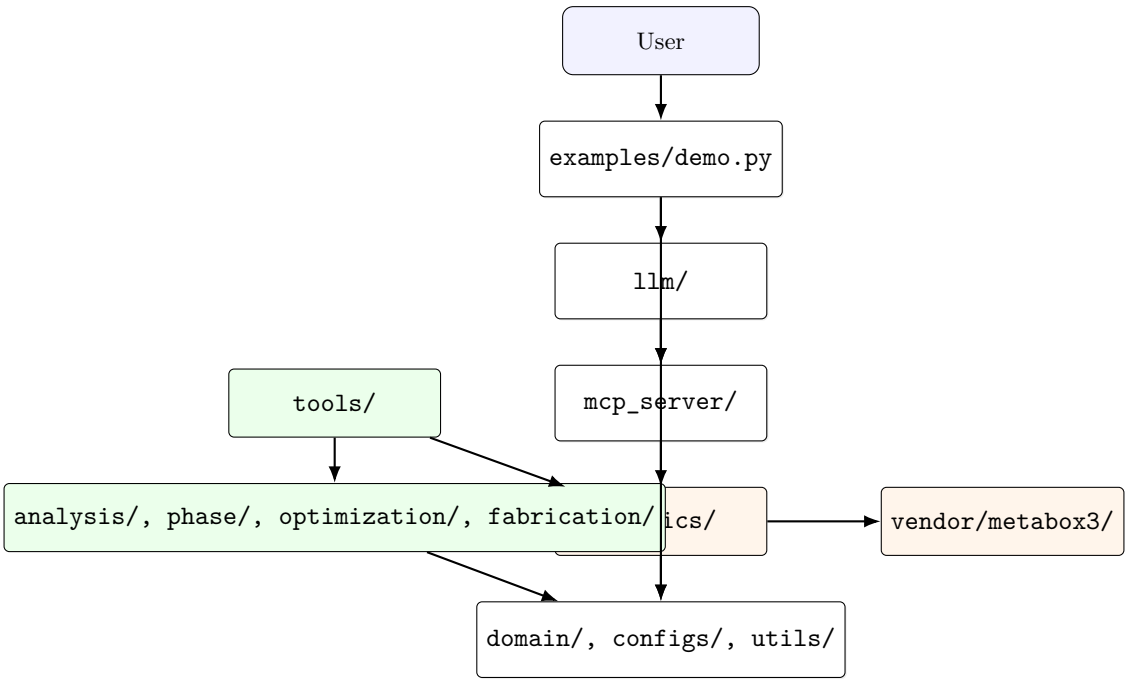


Figure 12.2: Package dependency graph for the shipping repository. The two routes from the top – demo via LLM, demo direct – share the MCP tool layer and the physics core.

Chapter 13

Software Engineering Analysis

13.1 Architecture

MetaCode's architecture follows the *ports and adapters* pattern. The scientific core is the hexagon:

- **Ports:** the SimulationBackend ABC and the ArtifactStore Protocol.
- **Adapters:** MetaboxBackend (metabox3), AnalyticalBackend (numpy only), S4Backend (stub), and the implicit in-memory store expected by every BaseTool with needs_store=True.
- **Driving sides:** llm/agent.py + mcp_server/tools.py, and the second src/metaopticsai/tools/ catalogue.

The strict rule that only physics/backends/metabox.py and physics/materials.py may import vendor.metabox3 keeps the TF dependency isolated. Everything else in the package runs on pure numpy / scipy / pydantic.

13.2 Modularity

Table 13.1 inventories the modules and the number of public callables.

Table 13.1: Modules and their public surface.

Module	Public symbols (approx.)
domain/*	11 types
configs/settings.py	1 singleton
physics/backends/*	1 ABC + 3 classes + registry
physics/rcwa.py, propagation.py, materials.py, fdtd.py	4 facades + FDTDStore + Entries
phase/*	4 functions
analysis/*	6 functions
optimization/*	1 ABC + 3 classes
fabrication/gds.py	1 class, lazy gdsfactory
tools/*	1 ABC + 1 Registry + 16 tools + 7 schema mod
mcp_server/*	6 handlers + TOOLS + dispatch
llm/*	run_agent, get_client, get_model_config, load_e
utils/*	setup_logging, dumps, safe_parse_llm_json, imag

13.3 Extensibility

Extensibility is unusually explicit. The code provides two documented extension points, each confined to a single file:

- Add a template: append a dict to TEMPLATES in mcp_server/templates.py.
- Add a tool: implement _tool_xxx(args), append the descriptor to TOOLS, register it in _HANDLERS - all in mcp_server/tools.py.

A third, less-documented extension point is the backend registry: register(backend) in physics/backends/__init__.py. Any class that subclasses SimulationBackend, implements is_available, and converts arrays to numpy before returning can be dropped in.

13.4 Error Handling

The repository is defensive at every layer:

- **Pydantic** enforces per-field and cross-field constraints at ingestion (no invalid dict reaches physics).
- **Facades** propagate Python exceptions upward.
- **Tool handlers** translate exceptions into {"ok": false, "error": "..."} with explicit except (ValidationError, KeyError, ValueError, RuntimeError) and a catch-all with a trimmed traceback.
- **Agent** catches JSON parse failures on function.arguments and converts them to the same tool-shape error so the LLM can read and recover.
- **Backends** degrade gracefully: a missing dependency returns False from is_available(); a caller selecting that backend sees a clean RuntimeError rather than an ImportError.

The one place with a soft-fail of concern is S4Backend's is_available(), which may return True if the user happens to have the S4 package installed; the next method call then raises NotImplementedError. See §15.

13.5 Logging

utils/logging.py::setup_logging() is idempotent and sets TensorFlow/absl/h5py/matplotlib to ERROR to suppress their default chattiness. Nothing in the current agent or MCP code calls setup_logging proactively - the demo relies on the Python default root logger plus the TF-silencing environment variable set by configs/settings.py. [INFERRED] A caller of the broader API should invoke setup_logging before heavy work.

13.6 Testing

[INFERRED] No visible tests/ directory exists in the inspected repository state. The dry-run mode of examples/demo.py doubles as a smoke test: it exercises every tool handler through dispatch without touching the network. A CI-style regression suite is not present.

13.7 Reproducibility

Three artifacts aid reproducibility:

- requirements.txt with pinned versions, including groq==0.37.1, tensorflow (expected via .[tf] extra - not visible in the main requirements file), pydantic_settings, gdsfactory==9. fastmcp==3.2.4.
- config/.env.example describing the environment variables (see also Table 6.1).
- Deterministic physics beneath dispatch() (modulo TF reductions).

What is *not* fixed:

- The LLM is non-deterministic even at $T = 0.1$; final text and the exact sequence of tool calls can vary across runs.
- numpy.random is *not* seeded by AnalyticalBackend; the backend's math is deterministic in any case.
- Metabox3's internal RNGs are not visibly seeded by MetaCode.

13.8 Performance

Rough mental model (not benchmarked for this report):

- **Analytical sweep (20 samples)**: dominated by Python overhead; sub-millisecond.
- **Metabox sweep (20 samples, 3x3 harmonics, resolution 64)**: hundreds of milliseconds to a few seconds depending on hardware; the mini-batch size setting (settings.rcwa.minibatch_si matches the 20-sample default.
- **Gradient optimisation (50 iters)**: minutes on CPU per iteration budget; practical on a modest GPU. The Adam learning rate is set in metres (2×10^{-9}) because metabox stores diameters in metres.
- **PSF/MTF/Strehl** (default 4× pad): a 2-D FFT of modest size; sub-second for anything the agent produces.
- **GDS write**: driven by gdsfactory; dominated by the number of unique diameters (cache hits) and the total unit-cell count. No benchmarks provided.

13.9 Security

The repository is a research prototype, so the attack surface is small, but a few observations:

- **API keys.** config/.env is git-ignored. load_env uses os.environ.setdefault, which prevents accidental overwrites of a key set externally.
- **Output path hardening.** ExportGDSTool resolves the target path against settings.paths.output_di and refuses any output_name that resolves outside it. The output_name field is Pydantic-constr with pattern=^[A-Za-z0-9_]{1,80}$.
- **Code execution.** The LLM cannot execute arbitrary code; it can only call the six registered handlers. Adding a handler is a code change, not an LLM-reachable action.

- **Input validation.** Pydantic bounds every numeric field; a free-form string reaching physics would be a bug.
- **Dependencies.** requirements.txt has pinned versions but is UTF-16 encoded (every character followed by a null byte in the dump). [INFERRED] This appears to be a Windows-produced file. Users should regenerate it via pip freeze after a working install.

13.10 Portability

- The repository was developed on Windows (git author dishantgupta2004; CRLF line endings in requirements.txt). No OS-specific paths are used in Python code; pathlib.Path is used throughout.
- TensorFlow availability is OS-dependent; the analytical fallback keeps the agent usable even where TF is not installable.
- gdsfactory is optional; _import_gdsfactory raises a clear ImportError if absent, with install instructions in the error message.

Chapter 14

Validation and Reproducibility

14.1 What Needs to Be Validated

An AI-orchestrated scientific tool has two distinct correctness burdens:

1. **Correctness of the computation.** Does MetaboxBackend.sweep_unit_cells compute the right complex transmission? Does AnalyticalBackend satisfy its stated approximation?
2. **Correctness of the agent's reportage.** Given a correct simulation result, does the final text summary faithfully reproduce the numbers the tool returned?

The two problems are decoupled by the "LLM never generates numbers" contract. Validation of (1) requires classical physics checks; validation of (2) requires a cross-check between the tool_calls log and the final text.

14.2 Validating the Physics Layer

Backend contract test (sketch). For a fixed MetalensDesignParams and IncidenceSpec, both MetaboxBackend.sweep_unit_cells and AnalyticalBackend.sweep_unit_cells must return arrays of the same shape and the same wavelength_m, and the metabox result should show qualitatively similar phase coverage behaviour. Quantitative agreement is *not* expected because the analytical form is a heuristic.

Phase monotonicity. Both backends zero-align phases (phases - phases[0]). The metabox path explicitly unwraps (np.unwrap). A reasonable test is that numpy.diff(phases) has at most a small fraction of negative entries over a sweep that spans $0 \to 2\pi$.

Material-table limits. get_material_index raises ValueError outside the CSV's tabulated range. A validation suite should probe each bundled material at the end-points of its valid range.

Analytical closed form. Eq. (2.2) is directly implemented; a unit test can assert that _phase_for_diameter(d_1)$-$_$phase_for_diameter(d_2)matchestheclosedformwithinfloating-pointtolerance.$

FFT-based analysis. compute_strehl_ratio on an unaberrated (uniform-phase) mask over its aperture must return $S = 1$ to within numerical precision. compute_psf on the same mask must have its peak at the geometric centre. These are good smoke tests.

14.3 Validating the Agent

Round-trip of a tool result. Compare `tool_calls[-1]["result"]` to the final text. The final text should quote `phase_coverage` and `mean_transmission` within rounding tolerance; any numeric claim that doesn't appear in *some* tool result is a hallucination.

Iteration-cap behaviour. Force `AGENT_MAX_ITERATIONS=1` and verify `stopped_on="iteration_cap"` and that the sentinel `final_text` string is returned.

Malformed tool arguments. Simulate a model that emits non-JSON `function.arguments`; verify that the tool-call log contains the `"Malformed JSON arguments..."` error and that the loop continues.

Determinism of `dispatch`. Call `dispatch` twice with identical arguments on the same backend; expect bit-identical scalar metrics for the analytical backend and hardware-dependent-but-close results for metabox (TF reductions can have small non-determinism on GPU).

14.4 Reproducibility Checklist

See also Appendix E for an operational version. Highlights:

- Pin the `GROQ_MODEL` version. The repository defaults to `llama-3.3-70b-versatile`, which Groq may update without notice.
- Fix `AGENT_MAX_ITERATIONS` and `GROQ_TEMPERATURE` to recorded values.
- Prefer `backend="metabox"` explicitly over `"auto"` for publication-grade runs; the `"auto"` fallback can silently switch to analytical.
- Capture the full `tool_calls` log (not just the final text) with every published result. Both the demo and the CLI (`-show-tools`) support this.
- Record `settings.rcwa.harmonics`, `settings.rcwa.resolution`, `settings.rcwa.minibatch_size` as explicit numbers with the result.
- Record Python version, numpy version, scipy version, TF version, metabox3 commit (vendored copy).

14.5 Known Reproducibility Hazards

- **LLM drift.** Groq-hosted models update periodically. A prompt that chose `tio2_metalens_532nm` today may choose a different template tomorrow. Mitigate by using strict prompts and recording the `tool_calls` log.
- **Partial fallback.** The `"auto"` backend silently selects analytical when TF is unavailable; results will differ sharply. Mitigate by inspecting `result.backend` in every logged call.
- **Non-seeded sampling in metabox.** Feature values are initialised with `tf.random.uniform(...)` when `initial_value` is None. The sweep path does not rely on this (it passes explicit sampled values), but the gradient optimisation path does. Mitigate by calling `tf.random.set_se` before gradient runs.

Chapter 15

Limitations

The limitations in this chapter are grounded in the repository as it is, not in opinions about what it ought to be. Where a limitation is inherent to the design intent (research prototype, single-agent, template-driven), that is said explicitly.

15.1 Current Limitations (A)

- [PARTIAL] **Only three templates ship.** TiO_2@532 nm, Si@850 nm, GaN@405 nm. Everything the LLM designs is a perturbation around one of these starting points. Mitigation: append entries to TEMPLATES with provenance notes.
- [PARTIAL] **Only diameter sweeps.** Both backend sweep_unit_cells methods raise NotImplementedError on any sweep_param other than "diameter".
- [PARTIAL] **One wavelength per sweep.** The IncidenceSpec admits a list of wavelengths, but every LLM-facing tool constructs the single-wavelength case.
- [PARTIAL] **S4 backend is a stub.** The three simulation methods raise NotImplementedError; its is_available() can return True if S4 happens to be installed, which creates a misleading state.
- [PARTIAL] **HeuristicOptimizer does not search.** Its optimize loop records the same Strehl estimate up to max_iters times. It is a scoring function, not an optimiser.
- [PARTIAL] **SurrogateOptimizer is a skeleton.** It returns FOMResult(value=0.0, metadata={"status":

15.2 Missing Capabilities (B)

- [FUTURE] No stdio MCP server. The ∼30-line change referenced in mcp_server/README.md is not shipped.
- [FUTURE] No per-call usage/cost reporting from Groq.
- [FUTURE] No persistent artifact store ships with the agent loop. The scientific tools layer expects an ArtifactStore Protocol but no concrete implementation is bundled with the LLM path.
- [FUTURE] No conversation-level memory beyond a single run_agent call.
- [FUTURE] No streaming of model output; each turn is a blocking create().
- [FUTURE] No parallel tool execution within one iteration (though the loop structure would permit it with trivial changes).

15.3 Architectural Limitations (C)

- [INFERRED] *Two parallel tool systems.* The mcp_server/tools.py registry (6 tools, dict-based, plain JSON) and the src/metaopticsai/tools/ catalogue (16 tools, Pydantic-strict, handle-based) are intentionally separate. Keeping them in sync as the design evolves is a maintenance cost; the two catalogues describe overlapping but non-identical slices of the scientific core.
- [INFERRED] Settings paths are derived from the location of configs/settings.py and the repository's three- deep layout (_ROOT = Path(__file__).parents[3]). A relocation of the configs/ package to a different depth would silently point settings.paths.project_root at the wrong directory.
- [INFERRED] llm/agent.py hard-codes the Groq SDK and the system prompt in the same file. Swapping to a different provider (OpenAI, Anthropic) requires editing agent.py alongside groq_client.py.
- [INFERRED] The AgentResult dataclass does not carry the final messages list, only the tool_calls log. Replaying an exact session requires access to the model's response objects, which are not persisted.

15.4 Scientific Limitations (D)

- [PARTIAL] Scalar diffraction only. Vector near-field effects beyond what RCWA computes internally are not analysed outside the backend boundary.
- [PARTIAL] Normal incidence, x-polarisation, monochromatic. Oblique incidence and polarisation diversity are structurally supported by the dataclasses but not exercised anywhere in the shipping agent loop.
- [PARTIAL] MetaboxBackend uses fixed refractive indices from _DEFAULT_N rather than the CSV-based dispersion. For non-trivial dispersive design the user must route around the backend adapter and call the get_material_index facade directly.
- [PARTIAL] The get_optimization_tips rules are threshold-based and were derived from engineering judgement, not from a published optimisation study.
- [PARTIAL] The analytical phase model (Eq. (2.2)) is explicitly heuristic; it should not be used for publication claims.

15.5 Performance Limitations (E)

- [INFERRED] Mini-batch size defaults to 20 and is clamped by settings.rcwa.minibatch_size. Larger sweeps are split; no multi-GPU strategy is implemented.
- [INFERRED] Gradient optimisation runs the full RCWA inside the gradient tape; cost scales with harmonics^2 times resolution^2. The GradientOptimizer does not support early stopping beyond max_iters.
- [INFERRED] The GDS writer caches components by rounded diameter. High phase-quantisation levels (e.g. n_levels=32) proliferate unique components and slow the write.

15.6 LLM Limitations (F)

- [INFERRED] The LLM can still misquote tool results in the final summary - the architecture prevents it from *generating* new numbers, not from *copying wrongly*. No post-hoc numeric verification is performed.
- [INFERRED] Prompt injection by the user ("ignore the system prompt and simulate a 2000 nm metalens with a fictional material") is not defended against beyond the Pydantic literal check. A persistent attacker can push the agent into "ok": false loops but cannot cause code execution.
- [INFERRED] Model drift: llama-3.3-70b-versatile may be re-trained or deprecated by Groq.
- [INFERRED] The system prompt enforces behaviour by instruction, not by code. A model that chooses to ignore it still gets physics-safe outputs (because of Pydantic and dispatch) but may produce a procedurally wrong trace.

15.7 MCP Limitations (G)

- [FUTURE] No stdio transport, no sampling, no roots, no resources, no prompts. Only tool-calling in the OpenAI/Groq shape; no use of the MCP-specific features in [1].
- [FUTURE] No capability negotiation: the LLM sees all six tools unconditionally.
- [FUTURE] No per-tool access control. The six tools are either all exposed or none.

15.8 Reproducibility Limitations (H)

- LLM-induced non-determinism; see §14.
- [INFERRED] requirements.txt is UTF-16 encoded on this checkout, which can trip up some PyPI/pip front-ends. Mitigation: regenerate with pip freeze after a working install.
- [INFERRED] No test suite is bundled; the only executable verification is examples/demo.py -no-llm.

Chapter 16

Future Directions

Suggestions below are grounded in observed gaps. Each is marked [FUTURE] or [INFERRED].

16.1 Short-Term Engineering Work

- [FUTURE] Ship a true stdio MCP server. The TOOLS descriptor list and dispatch() are already in the right shape; wrap them in fastmcp (already in requirements.txt) and expose the process through python -m mcp_server.
- [FUTURE] Expose the broader scientific tool catalogue over the same transport. A single build_tool_registry-backed server would enable phase-mask generation, PSF/MTF analysis, propagation, and GDS export from the LLM path. The existing BaseTool.call abstraction is MCP-ready (sync/async, Pydantic validation).
- [FUTURE] Add a persistent ArtifactStore implementation (filesystem-backed) alongside an in-process one. Both must satisfy the Protocol in tools/base.py; injecting them at the composition root does not touch the tool code.
- [FUTURE] Capture usage tokens from the Groq response and surface them in AgentResult for cost observability.
- [FUTURE] Add a numeric verification pass: before returning final_text, re-parse the agent's summary and cross-check every numeric claim against the tool_calls log. Mismatches become warnings.
- [FUTURE] Pin GROQ_MODEL to a specific dated snapshot (when Groq supports immutable aliases) and document it in config/.env.example.
- [FUTURE] Fix the UTF-16 requirements.txt; add a minimal pyproject.toml with the referenced [tool.importlinter] contract.

16.2 Short-Term Scientific Work

- [FUTURE] Add sweep axes beyond "diameter" to MetaboxBackend.sweep_unit_cells: period, height, duty-cycle, pillar-count-per-cell. The analytical backend currently refuses non-diameter sweeps explicitly.
- [FUTURE] Multi-wavelength sweeps. The IncidenceSpec already admits a list; the sweep infrastructure assumes a single wavelength downstream.

- [FUTURE] Oblique-incidence support in the agent-visible tools; the dataclass carries θ, ϕ already.
- [FUTURE] Fully implement S4Backend for cross-validation of RCWA against an independent implementation.
- [FUTURE] Expand the FDTD reference library in physics/fdtd.py and expose it through the LLM-facing toolbox (currently only via get_fdtd_library in the broader catalogue).
- [FUTURE] Replace the _DEFAULT_N dict in MetaboxBackend with calls to get_material_index, giving the LLM-driven path dispersive materials automatically.
- [FUTURE] Flesh out SurrogateOptimizer: the hooks to metabox3.modeling.load_metamodel are documented in the skeleton; the gradient-through-surrogate loop needs implementing.

16.3 Research Directions

- [FUTURE] **Inverse-design-in-agent.** Expose GradientOptimizer as an agent-callable tool with strict pre- and post-conditions, so the LLM can trigger a bounded gradient loop and interpret the resulting FOM history. Risk: the LLM must not propose unphysical learning_rate / max_iters without validation.
- [FUTURE] **Agentic phase-mask synthesis.** A two-tool extension pairs generate_phase_mask with an analysis call (PSF or MTF) and lets the LLM reason about the quantisation level that trades layout cost against aberration.
- [FUTURE] **Human-in-the-loop review.** Return a side-band "needs review" flag when the agent's chosen design has aspect_ratio>8 or NA> 0.8.
- [FUTURE] **Full-wavelength chromatic design.** Combine the multi-wavelength sweep with a cost that penalises chromatic defocus; the metabox3 assembly module (Metasurface, LensAssembly) supports wavelength lists.
- [FUTURE] **Design-rule-aware agents.** Encode fabricator-specific design rules (minimum feature size, minimum spacing) as hard pre-flight checks in a new validate_fabrication tool.
- [FUTURE] **Experimental closed loop.** Hook the agent to a measurement rig; let get_optimization_t be a learned model trained on past measurement/design mismatches rather than a static decision table.

16.4 Mapping Future Work to Current Files

Table 16.1 connects each suggestion to the files that would change. This is a convenience for readers who want to pick a starting point.

Table 16.1: Future-work entries and the files most likely to change.

Suggestion	**Primary file(s)**
stdio MCP server	new `mcp_server/server.py` + `__main__.py`
expose broader catalogue	new `mcp_server/scientific_tools.py`
persistent artifact store	new `src/metaopticsai/tools/store.py`
usage/cost reporting	`llm/agent.py` (append from `resp.usage`)
numeric verification pass	new helper in `llm/agent.py`
other sweep axes	`physics/backends/metabox.py`, `analytical.py`
multi-wavelength sweeps	`physics/rcwa.py`, backends
full `S4Backend`	`physics/backends/s4.py`
full `SurrogateOptimizer`	`optimization/surrogate.py`
dispersive materials in metabox	`physics/backends/metabox.py`
inverse-design-in-agent	`mcp_server/tools.py` + `optimization/gradient.py`

Chapter 17

Conclusion

MetaCode is a focused answer to a focused question: *how do we give an LLM access to a physics simulator without letting it make up physics?*

The answer, read off the code, has three parts. First, a strict Pydantic domain layer (MetalensDesignParams, IncidenceSpec) ensures that no numeric field ever reaches the RCWA solver without passing a range and cross-field check. Second, a small set of "verified" MCP-shaped tools - six of them - encode an explicit five-stage workflow, so the LLM's role shrinks from *design* to *selection among validated starting points*. Third, a pluggable backend registry gives the system a graceful TF-free fallback (AnalyticalBackend) so a developer without TensorFlow can still run the agent, with the active backend reported in every result.

The code is honest about what it is: a research prototype. Three templates, one sweep axis, one wavelength per sweep, no stdio MCP server, a stub S4 backend, a heuristic "optimiser" that doesn't actually search. But within those bounds the composition is coherent. Only two files anywhere in the repository import the vendored TensorFlow-based solver, and both do so lazily. The LLM sees a trimmed, scalar-only view of every result. The scientific tool layer (src/metaopticsai/tools/) is already structured as a transport-agnostic catalogue of BaseTools, waiting for a real MCP server to be bolted on.

For a researcher, the most useful thing MetaCode offers is a template to copy. The pattern of *Pydantic domain + ABC-defined backend + thin tool layer + LLM-driven procedure* composes cleanly, and the costs it imposes are modest: a system prompt, a decision table of thresholds, a dispatch map, and the discipline to make the LLM read numbers rather than invent them.

The gap between the shipping agent and the broader scientific tool catalogue (§5) is the single most interesting architectural edge of the project. Bridging it - by exposing the richer, handle-based toolbox over a real MCP server - would turn MetaCode from a template-selecti assistant into a reproducible, agentic meta-optics design environment. Everything upstream of that step is already written.

What remains for both the prototype and its possible successors is unglamorous: more templates, more sweep axes, a working S4 backend for cross-validation, dispersive materials in the metabox adapter, pinned model snapshots, and a test suite. None of these require revisions to the architecture.

Appendix A

Repository Tree

The listing below is *logical*: files that are not meaningful for architectural reading (byte-code caches, virtual environments, editor configuration) are omitted.

```
D:/metacode/
|-- README.md
|-- LICENSE
|-- requirements.txt
|-- .gitignore
|-- config/
| |-- .env.example <- template env file
| `-- .env <- git-ignored secrets
|-- llm/
| |-- __init__.py
| |-- __main__.py <- `python -m llm "..."`
| |-- agent.py <- SYSTEM_PROMPT, run_agent, AgentResult
| `-- groq_client.py <- load_env, get_client, get_model_config
|-- mcp_server/
| |-- __init__.py <- sys.path shim
| |-- README.md
| |-- tools.py <- TOOLS, _HANDLERS, dispatch
| |-- templates.py <- TEMPLATES, list_templates, get_template
| |-- workflow.py <- WORKFLOW_GUIDE text + accessor
| `-- simulator.py <- run_diameter_sweep wrapper
|-- src/
| `-- metaopticsai/
| |-- __init__.py <- __version__ = "0.2.0"
| |-- configs/
| | `-- settings.py <- RCWAConfig, PathsConfig, settings
| |-- domain/
| | |-- design.py <- MetalensDesignParams (Pydantic)
| | |-- handles.py <- Artifact, ArtifactKind
| | |-- incidence.py <- IncidenceSpec
| | |-- materials.py <- MaterialIndex
| | |-- results.py <- Unit/RCWA/Field/PSF/FOM dataclasses
| | `-- state.py <- DesignState (legacy GUI/notebooks)
| |-- physics/
| | |-- rcwa.py <- sweep_diameter facade
| | |-- propagation.py <- propagate facade
| | |-- materials.py <- CSV material lookup
| | |-- fdtd.py <- FDTD reference library
| | `-- backends/
| | |-- base.py <- SimulationBackend ABC + registry
| | |-- analytical.py <- AnalyticalBackend (numpy heuristic)
| | |-- metabox.py <- MetaboxBackend (TF, metabox3)
```

```
| | `-- s4.py <- S4Backend (stub)
| |-- phase/
| | |-- profiles.py <- FZL, axicon, SPP, from_image
| | `-- quantization.py <- quantize()
| |-- analysis/
| | |-- psf.py <- compute_psf
| | |-- mtf.py <- compute_mtf
| | |-- strehl.py <- compute_strehl_ratio
| | |-- zernike.py <- Noll-indexed decomposition
| | |-- encircled_energy.py <- cumulative-ring EE
| | `-- farfield.py <- Fraunhofer far-field
| |-- optimization/
| | |-- base.py <- Optimizer ABC
| | |-- heuristic.py <- HeuristicOptimizer (analytical FOM)
| | |-- gradient.py <- GradientOptimizer (TF via metabox3)
| | `-- surrogate.py <- SurrogateOptimizer (skeleton)
| |-- fabrication/
| | `-- gds.py <- GDSEngine (gdsfactory)
| |-- tools/
| | |-- __init__.py <- build_tool_registry
| | |-- base.py <- BaseTool, ToolRegistry, ArtifactStore
| | |-- rcwa_tools.py <- RunRCWASweepTool, GetFDTDLibraryTool
| | |-- phase_tools.py <- GeneratePhaseMaskTool
| | |-- analysis_tools.py <- AnalyzePSF/MTF/ZernikeTool
| | |-- propagation_tools.py <- PropagateFieldTool, intensity metrics
| | |-- materials_tools.py <- ListMaterials/GetMaterialIndexTool
| | |-- gds_tools.py <- ExportGDSTool
| | |-- optimization_tools.py <- OptimizeMetalens/TrainMetamodel tools
| | `-- schemas/ <- Pydantic I/O schemas (per-domain)
| |-- utils/
| | |-- logging.py <- setup_logging
| | |-- json_io.py <- NumpyJSONEncoder, safe_parse_llm_json
| | |-- async_utils.py <- to_async, maybe_await
| | `-- images.py <- figure/array -> base64 PNG
| `-- vendor/
| `-- metabox3/ <- vendored TF-based RCWA solver
| |-- __init__.py
| |-- utils.py <- Feature, Incidence, type aliases
| |-- rcwa.py <- Circle, Layer, UnitCell, SimConfig,
| | simulate_parameterized_unit_cells
| |-- propagation.py <- Field2D, get_transfer_function,
| | propagate
| |-- assembly.py <- Metasurface, LensAssembly,
| | optimize_single_lens_assembly
| |-- modeling.py <- sample_protocell, create_and_train_model
| |-- raster.py <- rasterisation primitives
| |-- expansion.py <- Fourier expansion helpers
| |-- export.py <- metabox-internal export helpers
| |-- metrics.py <- metabox-internal metrics
| |-- rcwa_tf/ <- TF kernels
| `-- material_data/*.csv <- 12 refractive-index CSVs
|-- examples/
| `-- demo.py <- dry-run / live-run demo
|-- notebooks/
| |-- Implementation.ipynb <- exploratory notebook (not analysed)
| `-- README.md
`-- docs/
    |-- METAOPTICS_REPORT.pdf <- pre-existing report
    `-- overleaf_latex/ <- this report
```

Appendix B

MCP Tool Reference

This appendix is a tool-by-tool reference. Section B.1 covers the six tools exposed to the LLM through mcp_server/ tools.py; section B.2 covers the broader src/metaopticsai/tools/ catalogue ([CONFIGURED]).

B.1 The Six LLM-Visible MCP Tools

B.1.1 `list_templates`

Purpose. Enumerate the three verified starting templates.

Input schema. None.

Output schema. {ok: bool, templates: [{name, summary, wavelength_nm, material, shape}]}

Implementation. mcp_server/tools.py::_tool_list_templates → templates.list_templates().

Called by. LLM in Stage 1.

Failure modes. None under normal operation.

B.1.2 `get_template`

Purpose. Return the full parameter dict for one template.

Input schema. {name: str}

Output schema. {ok: bool, template: {name, summary, wavelength_nm, material, shape, notes, params}}

Implementation. _tool_get_template → templates.get_template(name).

Failure modes. KeyError is caught and returned as {"ok":false, "error":"Unknown template ..."}.

B.1.3 `get_workflow_guide`

Purpose. Return the canonical five-stage workflow text.

Input schema. `None`.

Output schema. `{ok: bool, guide: str}`

Implementation. `_tool_get_workflow_guide` → `workflow.get_workflow_guide()`.

B.1.4 `validate_design`

Purpose. Pydantic-validate a candidate design and compute derived metrics.

Input schema. `{params: object}` where params is a MetalensDesignParams-shaped dict (see §5).

Output schema (success). `{ok: true, valid: true, errors: [], warnings: [...], derived: {numerical_aperture, f_number, aspect_ratio, mean_diameter_nm}}`

Output schema (failure). `{ok: true, valid: false, errors: [{field, msg}], warnings: []}`

Implementation. `_tool_validate_design`: instantiates `MetalensDesignParams(**params)`, catches `ValidationError`, appends two soft warnings on high aspect ratio or high NA.

Scientific role. The sole gatekeeper between untrusted LLM-produced dicts and physics.

B.1.5 `run_simulation`

Purpose. Run an RCWA diameter sweep through the registry.

Input schema.

```
{
  "params": {MetalensDesignParams fields},
  "n_samples": integer in [5, 60], default 20,
  "backend": "auto" | "metabox" | "analytical"
}
```

Output schema (LLM-visible).

```
{
  "ok": true,
  "result": {
    "backend", "sweep_param", "wavelength_nm",
    "phase_coverage", "mean_transmission", "elapsed_s",
    "n_samples",
    "sweep_head": {
       "sweep_values_nm": [first 5],
       "phases_rad": [first 5],
       "amplitudes": [first 5]
    }
  }
}
```

Internal implementation. simulator.run_diameter_sweep → physics.rcwa.sweep_diameter → backend sweep_unit_cells.

Failure modes. ValidationError, KeyError, ValueError, RuntimeError are each converted to {"ok":false,"error":"..."}; unexpected exceptions include a 3-level traceback.format_exc tail.

Hidden field. _full carries the raw 20-element vectors; stripped by llm/agent.py before being sent to the LLM.

B.1.6 get_optimization_tips

Purpose. Return rule-based textual guidance given a simulation result.

Input schema. {result: object} - the result subtree from run_simulation's output.

Output schema. {ok: bool, tips: [str, ...]}

Implementation. Three thresholds, four possible tips (see §10).

B.2 The Broader Scientific Tool Catalogue ([CONFIGURED])

The 16 tools below are declared in src/metaopticsai/tools/ and composed by build_tool_registry, but no shipping transport binds them to an LLM.

B.2.1 run_rcwa_sweep

Input (RCWASweepInput). wavelength_nm, pillar_material, pillar_height_nm, period_nm, min_diameter_nm max_diameter_nm, n_samples, xy_harmonics, backend.

Output (RCWASweepOutput). handle, phase_coverage_2pi_fraction, mean_transmission, elapsed_s, n_samples, backend, advice.

Internal. Dispatches to the BackendRegistry; stores the sweep in the artifact store with kind="rcwa_sweep".

B.2.2 get_fdtd_library

Input. None.

Output. {entries: [...], count: int} listing built-in wavelengths, shapes, materials, heights, periods, samples, and references.

Internal. Reads physics/fdtd.py::FDTDStore.

B.2.3 list_materials

Input. None.

Output. {materials: [str], count: int} from physics.materials.list_materials().

B.2.4 `get_material_index`

Input (`MaterialIndexInput`). `{name: str, wavelength_nm: float}` (bounds $(10, 50000)$ nm).

Output (`MaterialIndexOutput`). `{name, wavelength_nm, n, k, complex, valid_range_um}`.

B.2.5 `generate_phase_mask`

Input (`PhaseMaskInput`). `mask_type` $\in$ {`"fzl"`, `"axicon"`, `"spp"`}, plus `wavelength, diameter, pixel pitch, and the type-specific parameters.`

Output (`PhaseMaskOutput`). `{handle, shape, phase_min_rad, phase_max_rad, mask_type}`.

Internal. `Routes to the corresponding phase.profiles.* function; stores the mask with kind="phase_ma`

B.2.6 `analyze_psf`

Input (`PSFInput`). `{mask_handle, wavelength_nm, pixel_size_um, focal_length_um, pad_factor}`.

Output. `{handle, strehl_ratio, fwhm_x_um, shape, wavelength_nm}`.

B.2.7 `analyze_mtf`

Input. `{psf_handle, wavelength_nm, pixel_size_um, focal_length_um}`.

Output. `{handle, mtf50_lpmm, cutoff_lpmm}` plus stored MTF arrays.

B.2.8 `zernike_decompose`

Input. `{mask_handle, n_terms}` ($n_terms \in [4, 45]$).

Output. `{handle, top_aberrations: [{noll, name, coeff_rad}], n_terms_analyzed}`.

B.2.9 `propagate_field`

Input (`PropagateInput`). `{mask_handle, wavelength_nm, pixel_size_um, propagation_distance_um, refractive_index}`.

Output. `{handle, shape, peak_intensity, backend}`.

B.2.10 `compute_max_intensity` / `compute_center_intensity` / `compute_mtf_volume`

`Trio of scalar-field metrics, all consuming a psf_handle.`

B.2.11 `export_gds`

Input (`GDSExportInput`). `{design_handle, sweep_handle, output_name, period_nm, pixel_size_um, n_levels, shape, is_circular}` with output_name pinned by regex `^[A-Za-z0-9_]{1,80}$`.

Output. `{handle, path, size_kb, elapsed_s}`.

Internal. Resolves the final path against settings.paths.output_dir and refuses escapes; builds an FZL mask, quantises, hands to GDSEngine.generate().

B.2.12 optimize_metalens

Input (OptimizeInput). {sweep_handle, target_strehl, max_iters, learning_rate, method} where method ∈ {"heuristic","gradient","surrogate"}.

Output. {handle, achieved_strehl, iterations_run, target_met, method}.

Internal. Reconstructs a lightweight RCWASweepResult from the stored payload; chooses an optimiser (with a documented fallback to heuristic on any failure).

B.2.13 train_metamodel

Input (TrainMetamodelInput). Material, pillar height, period, min/max diameter, number of samples, number of epochs.

Output (JobSubmitOutput). {job_id, status: "queued", extra: {wavelength_nm, material}}.

Internal. Submits a background job via an injected job_manager; actual TF training runs in the worker (_train_metamodel_worker) which lazy-imports metabox3.rcwa, utils, modeling.

B.2.14 get_job

Input. {job_id: str}.

Output. {...status, result...} from the injected job_manager.

B.2.15 list_artifacts

Input. {kind?: ArtifactKind}.

Output. A list of handle summaries from the store.

Appendix C

Configuration

C.1 Environment Variables

Table C.1 is the complete list of environment variables read anywhere in the repository.

Table C.1: Environment variables.

Variable	Default	Where
GROQ_API_KEY	–	llm/groq_client.py::get_client
GROQ_MODEL	llama-3.3-70b-versatile	get_model_config
GROQ_TEMPERATURE	0.1	get_model_config
GROQ_MAX_TOKENS	1024	get_model_config
AGENT_MAX_ITERATIONS	5 (clamped to $[1, 5]$)	llm/agent.py::run_agent
RCWA_HARMONICS	3	configs/settings.py::RCWAConfig
RCWA_RESOLUTION	64	configs/settings.py::RCWAConfig
RCWA_MINIBATCH_SIZE	20	configs/settings.py::RCWAConfig
TF_CPP_MIN_LOG_LEVEL	3 (set by code)	configs/settings.py::Settings.__init__

C.2 config/.env.example

The checked-in template file:

```
GROQ_API_KEY=
GROQ_MODEL=llama-3.3-70b-versatile
GROQ_TEMPERATURE=0.1
GROQ_MAX_TOKENS=1024.
AGENT_MAX_ITERATIONS=5
```

Listing C.1: config/.env.example.

Note the stray . after 1024 in the shipping file; this is read-and-cast tolerant because int(float(...)) would accept "1024.", but the actual get_model_config uses int(os.environ.get("GROQ_ "1024")), which may *raise* on "1024.". Mitigation: edit your own config/.env to 1024.

C.3 Settings Singleton

configs/settings.py is loaded as a module-level singleton:

- settings.rcwa - RCWAConfig with the three RCWA env knobs.
- settings.paths.project_root - repository root, computed from settings.py's location.
- settings.paths.output_dir - auto-created on import, anchor for all tool outputs (GDS files, metamodels).
- settings.paths.papers_dir - reserved; not auto- created.
- settings.paths.material_dir - points at src/metaopticsai/vendor/metabox3/material_data/.

C.4 RCWA Knobs in Practice

- harmonics / xy_harmonics - 3 is the default; 5-7 is recommended for high-NA designs (noted in RCWASweepInput's field description).
- resolution - 64 is the default spatial grid in the unit cell. Higher values trade solve time for accuracy.
- minibatch_size - 20; used both by the sweep tool and the metamodel trainer.

C.5 Groq Model Choice

Any Groq-hosted tool-calling model works. The default llama-3.3-70b-versatile is a strong choice for structured tool use. Alternatives: llama-3.1-70b-versatile, llama-3.1-8b-instant (small, cheap, may need prompt reinforcement), or any later Groq tool-capable release.

Appendix D

API Reference

This appendix lists the *public* callables and classes grouped by package. Signatures reflect the shipping source.

D.1 llm/

D.1.1 llm.groq_client

- load_env(env_paths: Iterable[Path] $\|None = None) -> None$
- get_client() -> groq.Groq
- get_model_config() -> dict[str, Any]

D.1.2 llm.agent

- SYSTEM_PROMPT: str (module-level).
- @dataclass AgentResult: final_text, iterations, tool_calls, stopped_on.
- run_agent(user_prompt, max_iterations=None) -> AgentResult.

D.2 mcp_server/

D.2.1 mcp_server.tools

- TOOLS: list[dict] - JSON-Schema descriptors.
- dispatch(name, arguments) -> dict.
- list_available_backends() -> list[str].

D.2.2 mcp_server.templates

- TEMPLATES: dict[str, dict] - see §7.
- list_templates() -> list[dict].
- get_template(name: str) -> dict.

D.2.3 mcp_server.workflow

- WORKFLOW_GUIDE: str.
- get_workflow_guide() -> str.

D.2.4 mcp_server.simulator

- run_diameter_sweep(params, n_samples=20, backend="auto") -> dict.
- available_backends() -> list[str].

D.3 src/metaopticsai/

D.3.1 metaopticsai.domain

- class MetalensDesignParams(BaseModel) - see Table 5.1.
- class IncidenceSpec(BaseModel) with at_wavelength_nm(wl_nm).
- class MaterialIndex(BaseModel).
- class Artifact (frozen dataclass) + Artifact.new, .summary().
- ArtifactKind Literal.
- @dataclass UnitCellResult, RCWASweepResult, FieldResult, PSFResult, FOMResult.
- @dataclass DesignState.

D.3.2 metaopticsai.configs.settings

- settings (singleton) with rcwa, paths.

D.3.3 metaopticsai.physics

- rcwa.sweep_diameter(params, *, wavelength_nm=None, n=20, backend="auto") -> RCWASweepResult.
- rcwa.get_registry() -> BackendRegistry, rcwa.set_registry(reg).
- propagation.propagate(phase_mask, *, wavelength_nm, pixel_size_um, distance_um, refractive_inde backend="auto") -> FieldResult.
- materials.list_materials() -> list[str].
- materials.get_material_index(name, wavelength_nm) -> MaterialIndex.
- materials.get_material_csv_path(name) -> Path.
- fdtd.FDTDStore, BUILTIN_DATA, interpolate_phase_to_dimension.

D.3.4 metaopticsai.physics.backends

- class SimulationBackend(ABC) with is_available, simulate_unit_cell, sweep_unit_cells, propagate_field.
- class BackendRegistry with register, get(name="auto"), available, list_all.
- Concrete backends: AnalyticalBackend (name "analytical"), MetaboxBackend (name "metabox"), S4Backend (name "s4").
- default_registry() -> BackendRegistry.

D.3.5 metaopticsai.phase

- profiles.fresnel_zone_lens(...).
- profiles.axicon(...).
- profiles.spiral_phase_plate(...).
- profiles.from_image(filepath) -> np.ndarray.
- quantization.quantize(phase, n_levels, *, phase_min_deg=0, phase_max_deg=360) -> (ndarray, list).

D.3.6 metaopticsai.analysis

- psf.compute_psf(phase_mask, wavelength_nm, pixel_size_um, focal_length_um=0, pad_factor=4, amplitude=None).
- mtf.compute_mtf(psf_2d, pixel_size_um, wavelength_nm, focal_length_um=0).
- strehl.compute_strehl_ratio(phase_mask, pad_factor=4).
- zernike.zernike_polynomial(j, rho, theta), zernike.zernike_decomposition(phase_mask, n_terms=15), zernike.ZERNIKE_NAMES.
- encircled_energy.compute_encircled_energy, radius_for_fraction.
- farfield.compute_farfield.

D.3.7 metaopticsai.optimization

- class Optimizer(ABC) with optimize(params, *, target_fom, max_iters, **kwargs).
- class HeuristicOptimizer: requires sweep_result=RCWASweepResult.
- class GradientOptimizer: constructs a metabox3 LensAssembly and calls optimize_single_lens_ass with FigureOfMerit.LOG_STREHL_RATIO.
- class SurrogateOptimizer: skeleton.

D.3.8 metaopticsai.fabrication.gds.GDSEngine

- set_progress_callback(cb).
- generate(*, quantized_mask, bins, dimensions, phases, period, pixel_size_um, unitcells_per_pixe shape, is_circular, output_path, cross_width=0, fin_width=0, fin_length=0) -> dict.

D.3.9 metaopticsai.tools

- class ArtifactStore(Protocol).
- class BaseTool(ABC, Generic[InputT, OutputT]) with class attributes name, description, Input, Output, needs_store; methods call(raw) (async-aware), call_sync(raw), abstract _run(params).
- class ToolRegistry: register, get, list, names.
- build_tool_registry(store, backends, job_manager=None) -> ToolRegistry.

Per-tool class names and Pydantic schemas are documented in Appendix B.

D.3.10 `metaopticsai.utils`

- `logging.setup_logging(level=INFO, force=False)`, `get_logger(name)`.
- `json_io.dumps(obj, indent=2)`, `json_io.safe_parse_llm_json(text, fallback)`.
- `async_utils.to_async(fn, *args, **kwargs)`, `maybe_await(x)`.
- `images.fig_to_png_b64(fig, dpi=110)`, `array_to_png_b64(arr, ...)`, `chart_to_png_b64(plot_fn, ...)`.

Appendix E

Reproducibility Checklist

A compact operational checklist for reproducing MetaCode results.

E.1 Environment

- Python ≥ 3.10 with the standard scientific stack (numpy, scipy, Pillow, matplotlib). The vendored metabox3 declares Python 3.11.9+.
- pip install -r requirements.txt. Note: the shipping requirements.txt is UTF-16 encoded on this checkout; regenerate with pip freeze after a working install.
- TensorFlow is required for the metabox backend; absence falls back to the analytical backend.
- gdsfactory is required only for GDS export.
- A valid GROQ_API_KEY is required for live runs.

E.2 Fixing the Agent

- GROQ_MODEL=llama-3.3-70b-versatile (or an immutable snapshot).
- GROQ_TEMPERATURE=0.1.
- GROQ_MAX_TOKENS=1024.
- AGENT_MAX_ITERATIONS=5.

Record the exact values alongside the result.

E.3 Fixing the Physics

- Specify backend="metabox" explicitly for publication-grade runs. Verify in each tool call that result.backend == "metabox".
- Record settings.rcwa.harmonics (default 3, xy_harmonics=(3,3) unless overridden by MetalensDesignParams), settings.rcwa.resolution (64), and settings.rcwa.minibatch_size (20).
- If using the gradient optimizer: set tf.random.set_seed(k) before the call; record k, the Adam learning rate, and max_iters.

E.4 Capturing the Session

- Run via `python examples/demo.py -prompt "..."` or `python -m llm "..." -show-tools` so the per-call trace is printed.
- Save stdout to a file; the trace is JSON-encodable but is printed as human-readable blocks.
- For programmatic captures:

```python
from llm.agent import run_agent
r = run_agent("...")
record = {
    "final_text": r.final_text,
    "iterations": r.iterations,
    "stopped_on": r.stopped_on,
    "tool_calls": r.tool_calls, # includes "_full" where present
}
import json
with open("run.json", "w") as f:
    json.dump(record, f, default=str, indent=2)
```

Listing E.1: Programmatic session capture.

E.5 Verifying Numeric Faithfulness

- Open `run.json`; read `tool_calls[-1]["result"]` (the final tool result the LLM saw).
- Grep `final_text` for the reported numbers. Each numeric claim should match a value in *some* tool-call result.
- If a numeric claim is *not* in any tool-call result, flag it as a hallucination; the architecture does not currently guard against this (see §15).

E.6 Minimal Reproducer Script

```python
from mcp_server.tools import dispatch

# 1. Pick a template
template = dispatch("get_template",
                    {"name": "tio2_metalens_532nm"})["template"]
params = template["params"]

# 2. Validate
v = dispatch("validate_design", {"params": params})
assert v["valid"], v["errors"]

# 3. Simulate (force metabox)
r = dispatch("run_simulation",
             {"params": params, "n_samples": 20, "backend": "metabox"})
assert r["ok"], r.get("error")

```

```
# 4. Interpret
print(r["result"]["phase_coverage"], r["result"]["mean_transmission"])
```

Listing E.2: A minimal reproducer bypassing the LLM.

If this script runs clean on your checkout, the physics layer is reproducible; any additional non-determinism arises in the LLM path.

Bibliography

[1] Anthropic. Model context protocol: Specification and reference. https://modelcontextprotocol.io/, 2024. Specification referenced for JSON-Schema tool shapes used by MetaCode's mcp_server package.

[2] Max Born and Emil Wolf. *Principles of Optics*. Cambridge University Press, 7th edition, 1999.

[3] Joseph W. Goodman. *Introduction to Fourier Optics*. W. H. Freeman, New York, 4th edition, 2017.

[4] Chen Huang and collaborators. *metabox3*: a tensorflow-based rcwa simulator and metasurface design toolbox. Software library vendored under src/metaopticsai/vendor/metabox3/, 2025.

[5] Mohammadreza Khorasaninejad, Wei Ting Chen, Robert C. Devlin, Jaewon Oh, Alexander Y. Zhu, and Federico Capasso. Metalenses at visible wavelengths: Diffraction-limited focusing and subwavelength resolution imaging. *Science*, 352(6290):1190–1194, 2016.

[6] Dianmin Lin, Pengyu Fan, Erez Hasman, and Mark L. Brongersma. Dielectric gradient metasurface optical elements. *Science*, 345(6194):298–302, 2014.

[7] Victor Liu and Shanhui Fan. S4: A free electromagnetic solver for layered periodic structures. *Computer Physics Communications*, 183(10):2233–2244, 2012.

[8] André Maréchal. étude des effets combinés de la diffraction et des aberrations géométriques sur l'image d'un point lumineux. *Revue d'Optique*, 26:257–277, 1947.

[9] M. G. Moharam, Eric B. Grann, Drew A. Pommet, and T. K. Gaylord. Formulation for stable and efficient implementation of the rigorous coupled-wave analysis of binary gratings. *Journal of the Optical Society of America A*, 12(5):1068–1076, 1995.

[10] Robert J. Noll. Zernike polynomials and atmospheric turbulence. *Journal of the Optical Society of America*, 66(3):207–211, 1976.

[11] Nanfang Yu, Patrice Genevet, Mikhail A. Kats, Francesco Aieta, Jean-Philippe Tetienne, Federico Capasso, and Zeno Gaburro. Light propagation with phase discontinuities: Generalized laws of reflection and refraction. *Science*, 334(6054):333–337, 2011.